MLPs for NLP
RNNs, LSTMs

M. Vazirgiannis

MBZUAI &
DASCIM, LIX École Polytechnique
Google Scholar: https://bit.ly/2rwmvQU

September, 2026

Outline

1. MLP applications to NLP
   - MLPs for Word embeddings
2. Recurrent NNs + LSTMs

Language model

- Goal: determine $P(s = w_1 \dots w_k)$ in some domain of interest

$$P(s) = \prod_{i=1}^{k} P(w_i | w_1 \dots w_{i-1})$$

  e.g. $P(w_1 w_2 w_3) = P(w_1) P(w_2|w_1) P(w_3|w_1 w_2)$

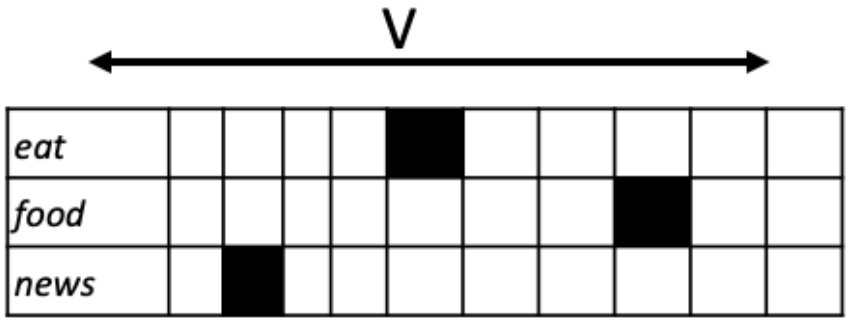
- Traditional n-gram language model assumption: "the probability of a word depends only on context of n-1 previous words"

$$\Rightarrow \hat{P}(s) = \prod_{i=1}^{k} P(w_i | w_{i-n+1} \dots w_{i-1})$$

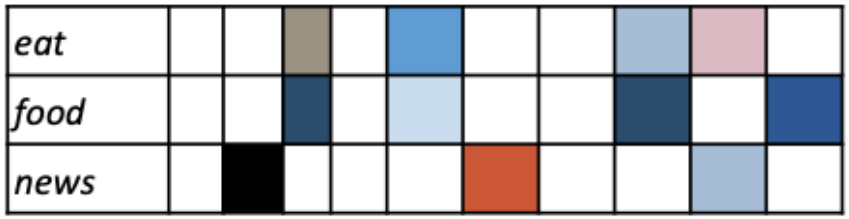
- Typical ML-smoothing learning process (e.g., Katz 1987):
  1. compute $\hat{P}(w_i | w_{i-n+1} \dots w_{i-1}) = \frac{\# w_{i-n+1} \dots w_{i-1} w_i}{\# w_{i-n+1} \dots w_{i-1}}$ on training corpus
  2. smooth to avoid zero probabilities

Representing Words

- One-hot vector
  - high dimensionality
  - sparse vectors
  - dimensions=$|V|$ ($10^6 < |V|$)
  - unable to capture semantic similarity between words



- Distributional vector
  - words that occur in similar contexts, tend to have similar meanings
  - each word vector contains the frequencies of all its neighbors
  - dimensions=$|V|$
  - computational complexity for ML algorithms



Example

- We should assign similar probabilities (discover similarity) to *Obama speaks to the media in Illinois* and the *President addresses the press in Chicago*
- This does not happen because of the "one-hot" vector space representation

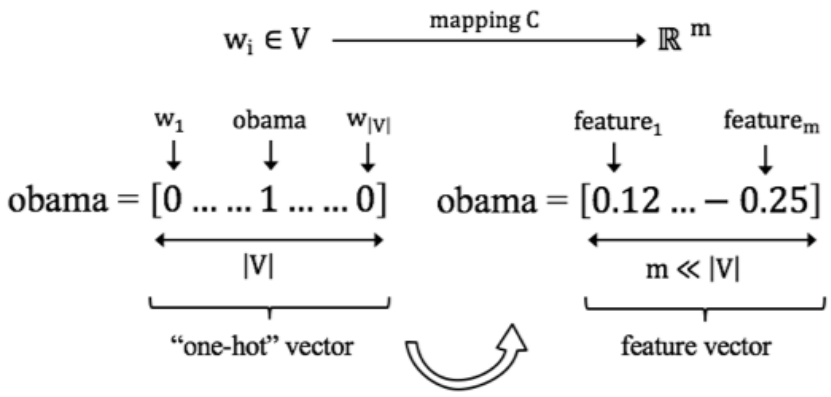
One hot

$$\text{obama} = [0\,0\,0\,0\ \ldots\ 0\,1\,0\,0] \quad \text{president} = [0\,0\,0\,1\ \ldots\ 0\,0\,0\,0] \;\Rightarrow\; \overrightarrow{\text{obama}}.\overrightarrow{\text{president}} = \vec{0}$$

$$\text{speaks} = [0\,0\,1\,0\ \ldots\ 0\,0\,0\,0] \quad \text{addresses} = [0\,0\,0\,0\ \ldots\ 0\,0\,1\,0] \;\Rightarrow\; \overrightarrow{\text{speaks}}.\overrightarrow{\text{addresses}} = \vec{0}$$

$$\text{illinois} = [1\,0\,0\,0\ \ldots\ 0\,0\,0\,0] \quad \text{chicago} = [0\,1\,0\,0\ \ldots\ 0\,0\,0\,0] \;\Rightarrow\; \overrightarrow{\text{illinois}}.\overrightarrow{\text{chicago}} = \vec{0}$$

Word Embedding



Co-occurrence matrix

- I like to fly
- I enjoy learning to fly
- I like learning english

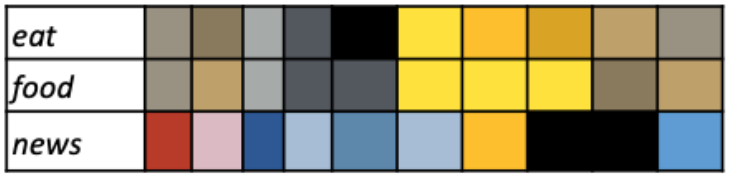
	I	like	fly	enjoy	learning	english
I		1	1	1	1	1
like	1		1		1	1
fly	1			1		
enjoy	1		1		1	
learning	1	1	1			1
english	1	1			1	

- The matrix is large and sparse
  - Storage
  - Classification
- Learn *a low dimensional dense representation of words*

Representing Words

Word embedding

- store the same contextual information in a low-dimensional vector
- **densification** (sparse to dense)
- **compression**
  - dimensionality reduction
  - dimensions=m
    $100 < m < 500$
- able to capture semantic similarity between words
- learned vectors (unsupervised)
- Learning methods
  - SVD
  - word2vec
  - GloVe

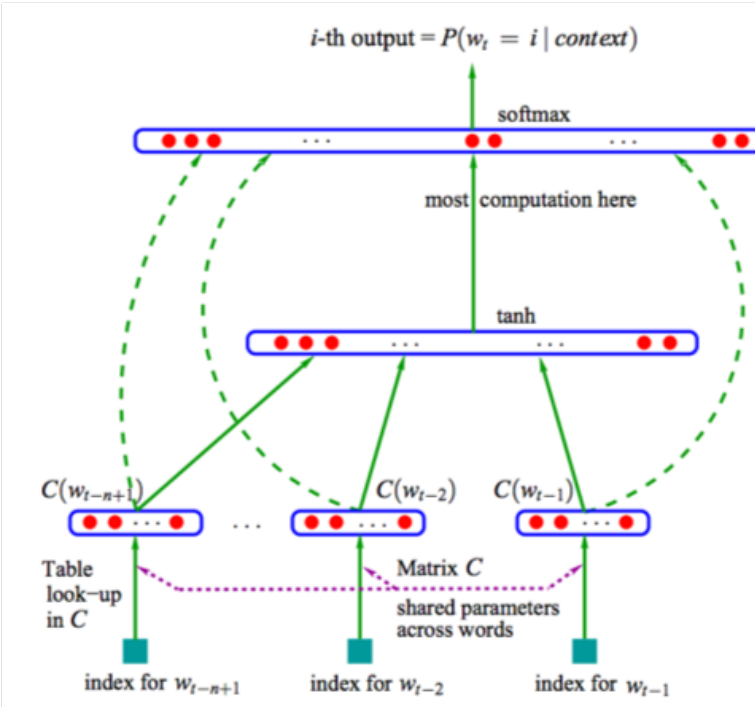


Learning Word Vectors

1. associate with each word in the vocabulary distributed *word feature* vector (a real-valued vector in $\mathbb{R}^m$)
2. express the joint *probability function* of word sequences in terms of the feature vectors of these words in the sequence, and
3. learn simultaneously the *word feature vectors* and the parameters of that *probability function*

- Corpus containing T training words
- Objective :
  $f(w_t, \ldots, w_{t-n+1}) = \hat{P}(w_t | w_{t-n+1}, \ldots, w_{t-1})$
- Decomposed in two parts: $w_i \in V \xrightarrow{\text{mapping C}} \mathbb{R}^m$
  - Mapping **C** (1-hot v $\Rightarrow$ lower dimensions)
  - Mapping any **g** s.t. estimate prob $t+1|t$ previous)

  $f(w_{t-1}, \ldots, w_{t-n+1}) =$
  $g(C(w_{t-1}), \ldots, C(w_{t-n+1}))$
- C(i): i-th word feature vector (Word embedding)
- Objective function: $J = \frac{1}{T} \sum f(w_t, \ldots, w_{t-n+1})$

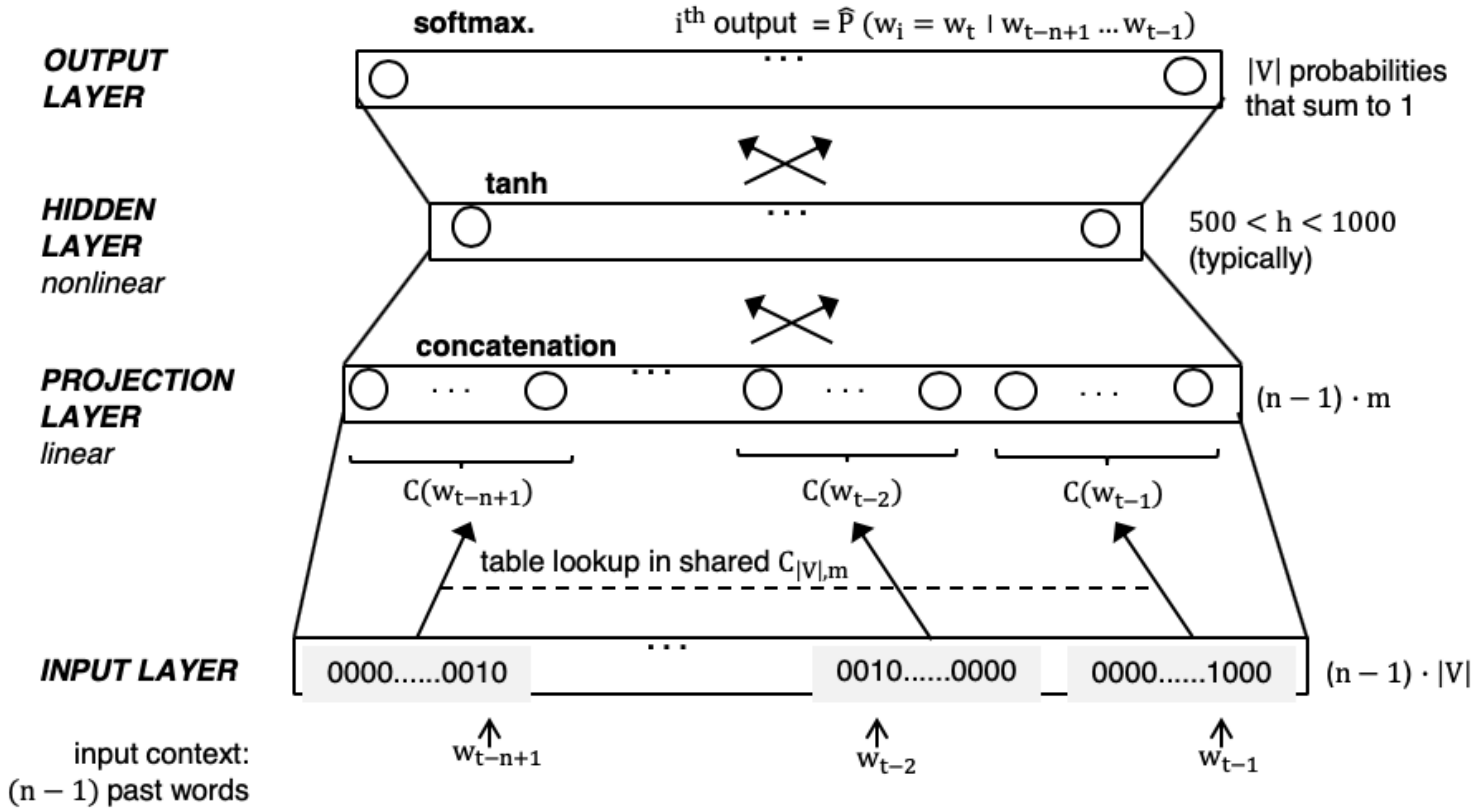


Bengio, Yoshua, et al. "A neural probabilistic language model." JMLR 3 (2003): 1137-1155.

Neural Net Language Model

For each training sequence:

- input: (context, target)
- pair: $(w_{t-n+1}, \ldots, w_{t-1}, w_t)$
- objective: minimize $E = -\log \hat{P}(w_t | w_{t-n+1}, \ldots, w_{t-1})$



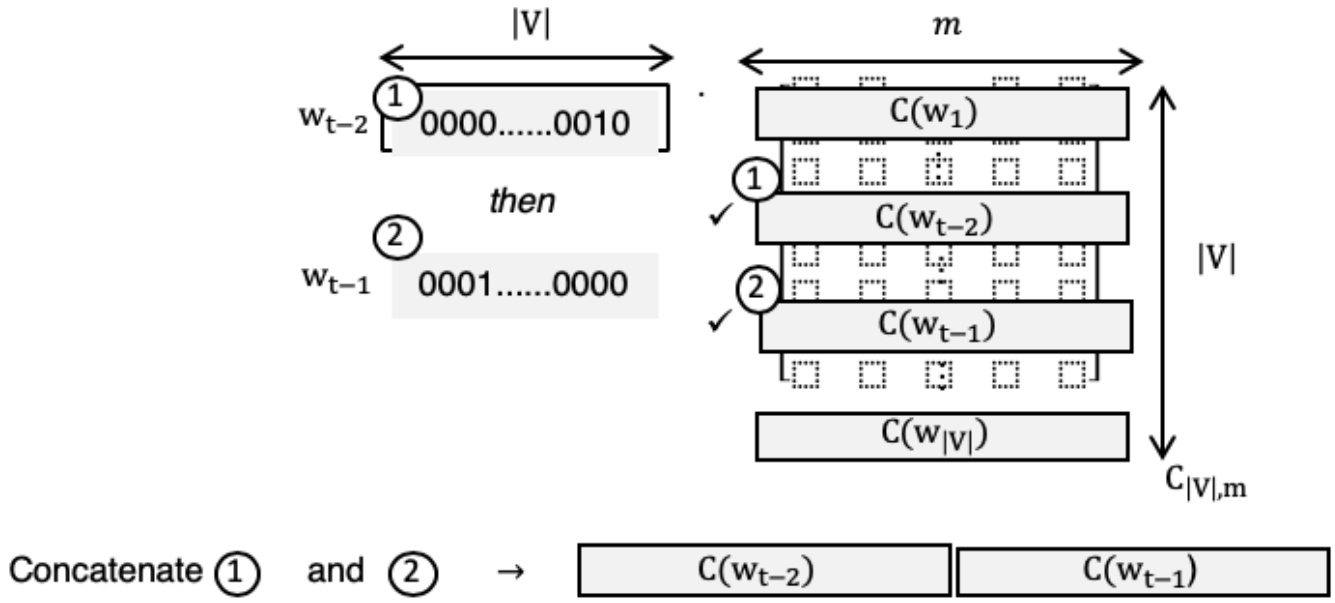
Objective function

- $E = -\log \hat{P}(w_t | w_{t-n+1}, \ldots, w_{t-1})$
- a probability between 0 and 1.
- On this support, the log is negative $\Rightarrow$ –log term positive.
- makes sense to try to minimize it.
  - Probability of word given the context be as high as possible (1 for a perfect prediction).
  - case the error is equal to 0 (global minimum).

p	log(p)	-log(p)
0.7	-0.15490196	0.15490196
0.2	-0.698970004	0.698970004

NNLM Projection layer

- Performs a simple table lookup in $C_{|V|,m}$: concatenate the rows of the shared mapping matrix $C_{|V|,m}$ corresponding to the context words. Example for a two-word context w_{t-2} w_{t-1}:



- $C_{|V|,m}$ is **critical**: it contains the weights that are tuned at each step. After training, it contains what we're interested in: the **word vectors**

Bengio, Yoshua, et al. "A neural probabilistic language model." JMLR 3 (2003): 1137-1155.

NNLM hidden/output layers and training

- Softmax (log-linear classification model) outputs positive numbers that sum to one (a multinomial probability distribution): i^{th} unit in the output layer:
  $\hat{P}(w_i = w_t | w_{t-n+1}, \ldots, w_{t-1}) = \frac{e^{y_{w_i}}}{\sum_{i'=1}^{|V|} e^{y_{w_{i'}}}}$

  where

  - $y = b + U \cdot \tanh(d + H \cdot x)$
  - tanh: nonlinear squashing (link) function
  - x : concatenation C(w) of the context weight vectors seen previously
  - b : output layer biases ($|V|$ elements)
  - d : hidden layer biases (h elements). Typically $500 < h < 1000$
  - U: $|V| \times h$ matrix storing the hidden-to-output weights
  - H: $h \times (n-1)m$ matrix storing the projection-to-hidden weights

  $\boldsymbol{\theta = (b, d, U, H, C)}$
  complexity per training sequence: $n * m + n * m * h + \boldsymbol{h * |V|}$
  computational bottleneck: **nonlinear hidden layer** ($h * |V|$ term)

- Training performed via stochastic gradient descent (learning rate ϵ):

$$\theta \leftarrow \theta + \epsilon \frac{\partial E}{\partial \theta} = \theta + \epsilon \frac{\partial \log \hat{P}(w_t | w_{t-n+1}, \ldots, w_{t-1})}{\partial \theta}$$

  (weights are initialized randomly, then updated via backpropagation)

NNLM facts

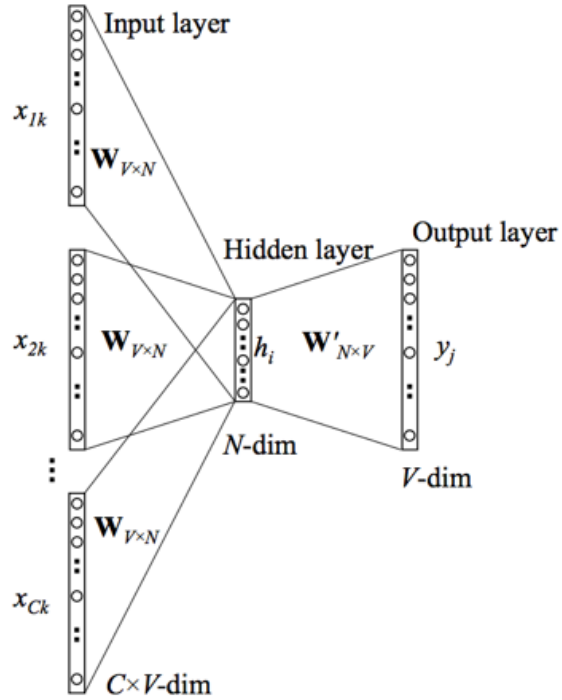
- tested on Brown (1.2M words, $|V| \cong 16K$) and AP News (14M words, $|V| \cong 150K$ reduced to 18K) corpuses
- h: # hidden units, n= # words in context, m: dimensionality of word vectors.
- Brown: h=100, n=5, m=30
- AP News: h=60, n=6, m=100, **3 week** training using **40 cores**
- 24% and 8% relative improvement (resp.) over traditional smoothed n-gram LMs
- in terms of *test set perplexity*: geometric average of $1/\hat{P}(w_t|w_{t-n+1}, \ldots, w_{t-1})$
- Due to **complexity**, NNLM can't be applied to large data sets $\rightarrow$ poor performance on rare words
- Bengio et al. (2003) claims main contribution was a more accurate LM. They let the interpretation and use of the word vectors as **future work**
- On the opposite, Mikolov et al. (2013) focus on the **word vectors**

Word2Vec

- Mikolov et al. in 2013
- Key idea of word2vec: achieve better performance not by using a more complex model (i.e., with more layers), but by allowing a **simpler (shallower) model** to be trained on **much larger amounts of data**
- no hidden layer (leads to 1000 $\times$ speedup)
- projection layer is shared (not just the weight matrix) - C
- context: words from both history & future:
- Two algorithms for learning words vectors:
  - **CBOW**: from context predict target
  - **Skip-gram**: from target predict context

CBOW

- continuous bag-of-words
- continuous representations whose order is of no importance
- uses the surrounding words to predict the center word
- n-words before and after the target word

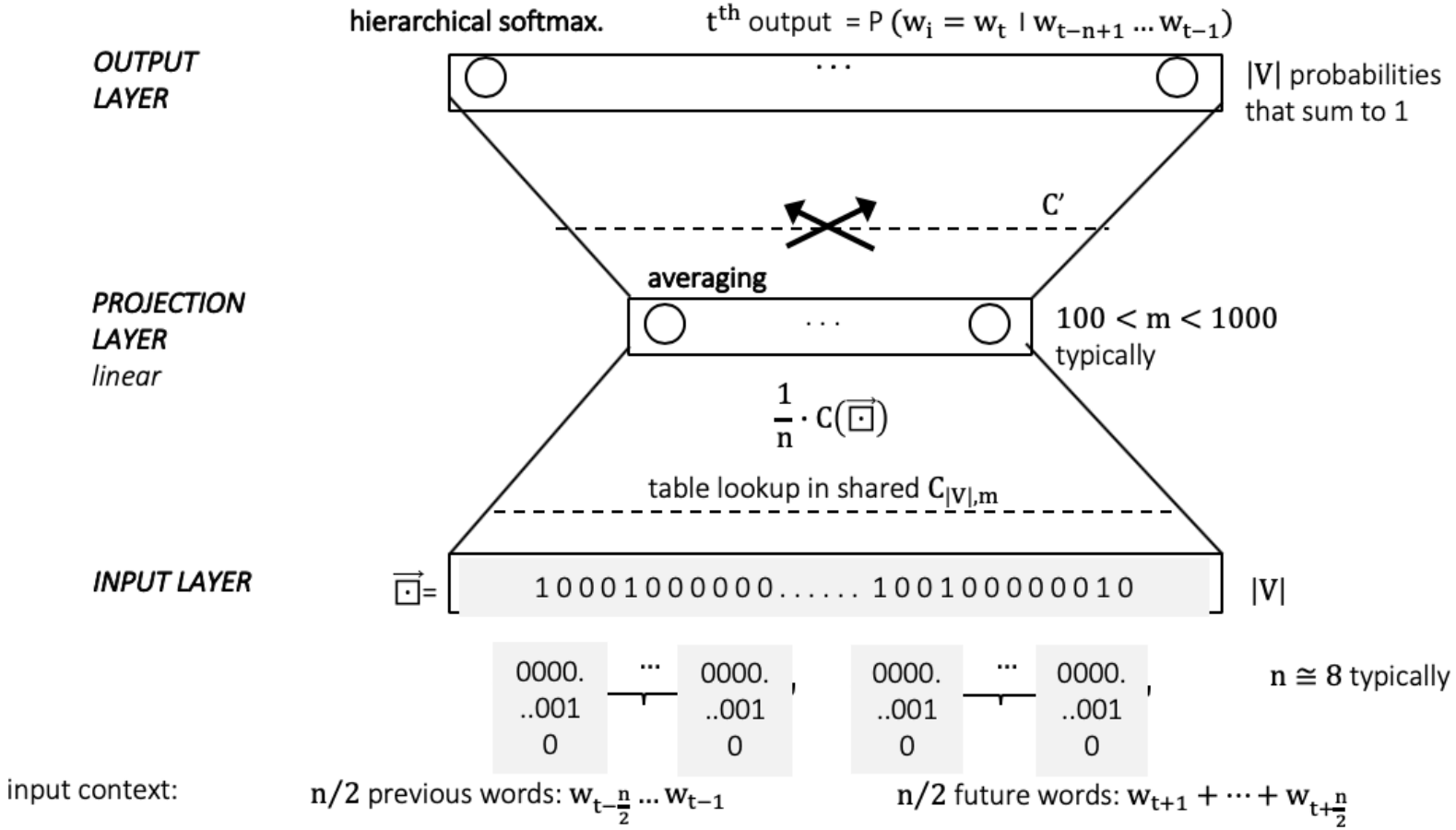


Efficient Estimation of Word Representations in Vector Space- Mikolov et al.

Continuous Bag-of-Words (CBOW)

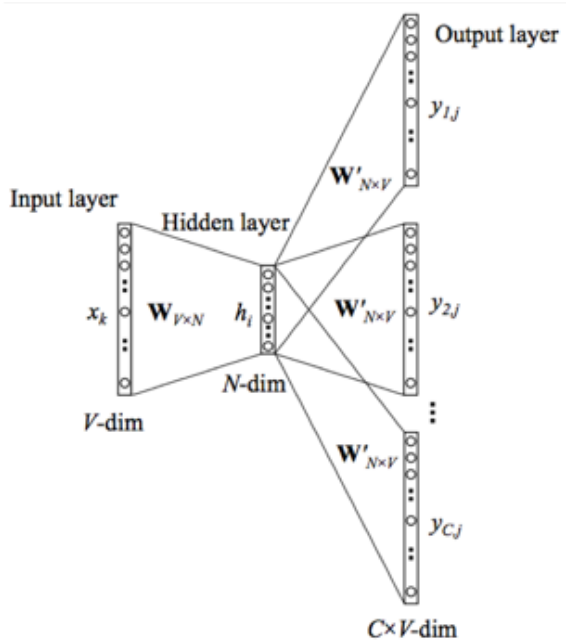
For each training sequence:

- input = (context, target)
- pair: $(w_{t-\frac{n}{2}}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+\frac{n}{2}}, w_t)$
- minimize $-\log \hat{P}(w_t | w_{t-n+1}, \ldots, w_{t-1})$



Skip-gram

- skip-gram uses, in a context, the center word to predict the surrounding words
- instead of computing the probability of the target word w_t given its previous words, we calculate the probability of the surrounding word w_{t+j} given w_t
- $P(w_{t+j}|w_t) = \frac{\exp(v_{w_t}^T v'_{w_{t+j}})}{\sum_{w \in V} \exp(v_{w_t}^T v'_{w_{t+j}})}$
- $v_{w_t}^T$ is a column of W_{VxN} and ${v'_{w_{t+j}}}^T$ is a column of W'_{NxV}
- objective function $J = \frac{1}{T} \sum_{t=1}^{T} \sum_{-n \leqslant j \leqslant n} \log p(w_{t+j}|w_t)$



Efficient Estimation of Word Representations in Vector Space- Mikolov et al. 2013

word2vec facts

- Complexity is $\boldsymbol{n * m + m * \log |V|}$ (Mikolov et al. 2013a)
- n: size of the context window (~ 10), $\boldsymbol{n \times m}$: dimensions of the projection layer, $\boldsymbol{|V|}$: size of the vocabulary
- On Google news 6B words training corpus, with $|V| \sim 10^6$:
  - CBOW with m=1000 took **2 days** to train on **140 cores**
  - Skip-gram with m=1000 took **2.5 days** on **125 cores**
  - NNLM (Bengio et al. 2003) took **14 days** on **180 cores**, for m=100 only! (m=1000 was not feasible)
- word2vec training *speed* $\cong 100K - 5M$ words/s
- Quality of the word vectors:
  - $\uparrow$ significantly with **amount of training data** and **dimension of the word vectors** (m), with diminishing relative improvements
  - measured in terms of accuracy on 20K semantic and syntactic association tasks:

Capital-Country	Past tense	Superlative	Male-Female	Opposite
Athens: **Greece**	walking: **walked**	easy: **easiest**	brother: **sister**	ethical: **unethical**

- Best NNLM: 12.3% overall accuracy. Word2vec (with Skip-gram): 53.3%

---

http://www.scribd.com/doc/285890694/NIPS-DeepLearningWorkshop-NNforText#scribd

https://code.google.com/p/word2vec/

Glove

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k\|ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k\|steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k\|ice)/P(k\|steam)$	8.9	8.5×10^{-2}	1.36	0.96

- Ratio of co-occurrence probabilities best distinguishes relevant words
  $F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}} \Rightarrow w_i^T \tilde{w}_k + b_i + \tilde{b}_k = \log(X_{ik})$
- Cast this into a least square problem: $J = \sum_{i,j=1}^{V} f(X_{ij})(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$
- X: co-occurrence matrix
- f: weighting function $f(x) = \begin{cases} (x/x_{\max})^\alpha & \text{if } x < x_{\max} \\ 1 & \text{otherwise} \end{cases}$
- b: bias terms
- w_i = word vector
- $\tilde{w}_j$ = context vector

model that utilizes

- count data
- bilinear prediction-based methods like word2vec

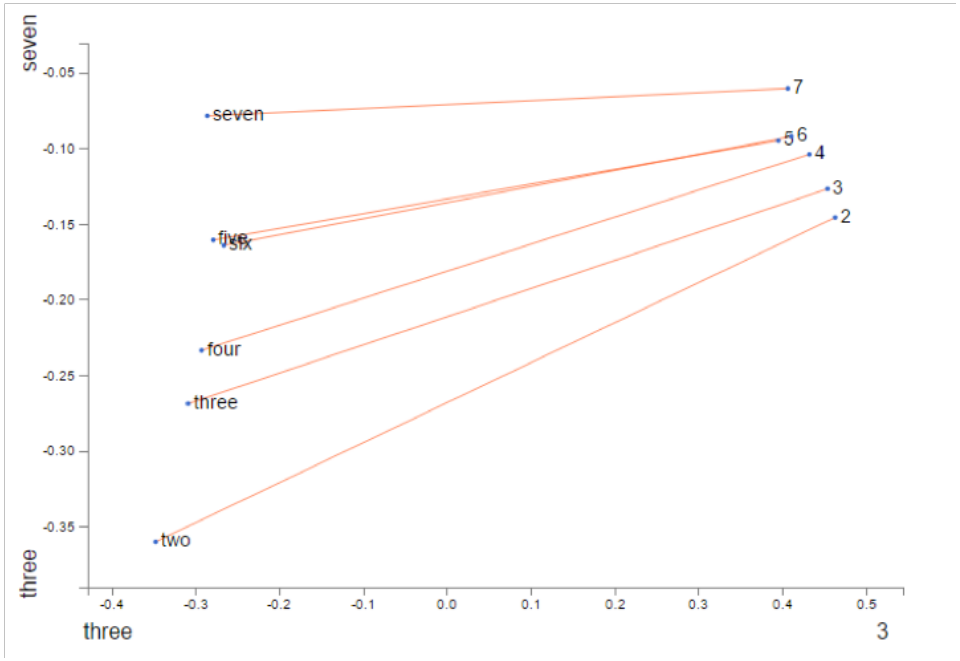
https://nlp.stanford.edu/pubs/glove.pdf

Which is better?

- Open question: SVD vs word2vec vs GloVe (all based on co-occurrence)
- Levy, O., Goldberg, Y., & Dagan, I. (2015)
  - SVD performs best on similarity tasks
  - Word2vec performs best on analogy tasks
  - No single algorithm consistently outperforms the other methods
  - Hyperparameter tuning is important
  - 3 out of 6 cases, tuning hyperparameters is more beneficial than increasing corpus size
  - word2vec outperforms GloVe on all tasks
  - CBOW is worse than skip-gram on all tasks

Applications

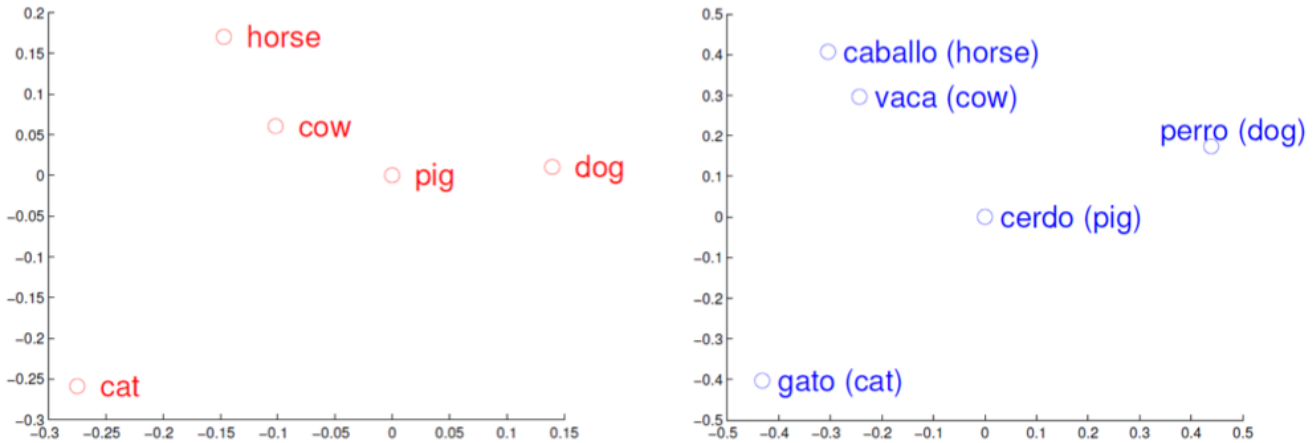
- Word analogies
- Find similar words (semantic & syntactic)
- POS tagging
- Similar analogies for different languages
- Document classification



https://lamyiowce.github.io/word2viz/

Applications

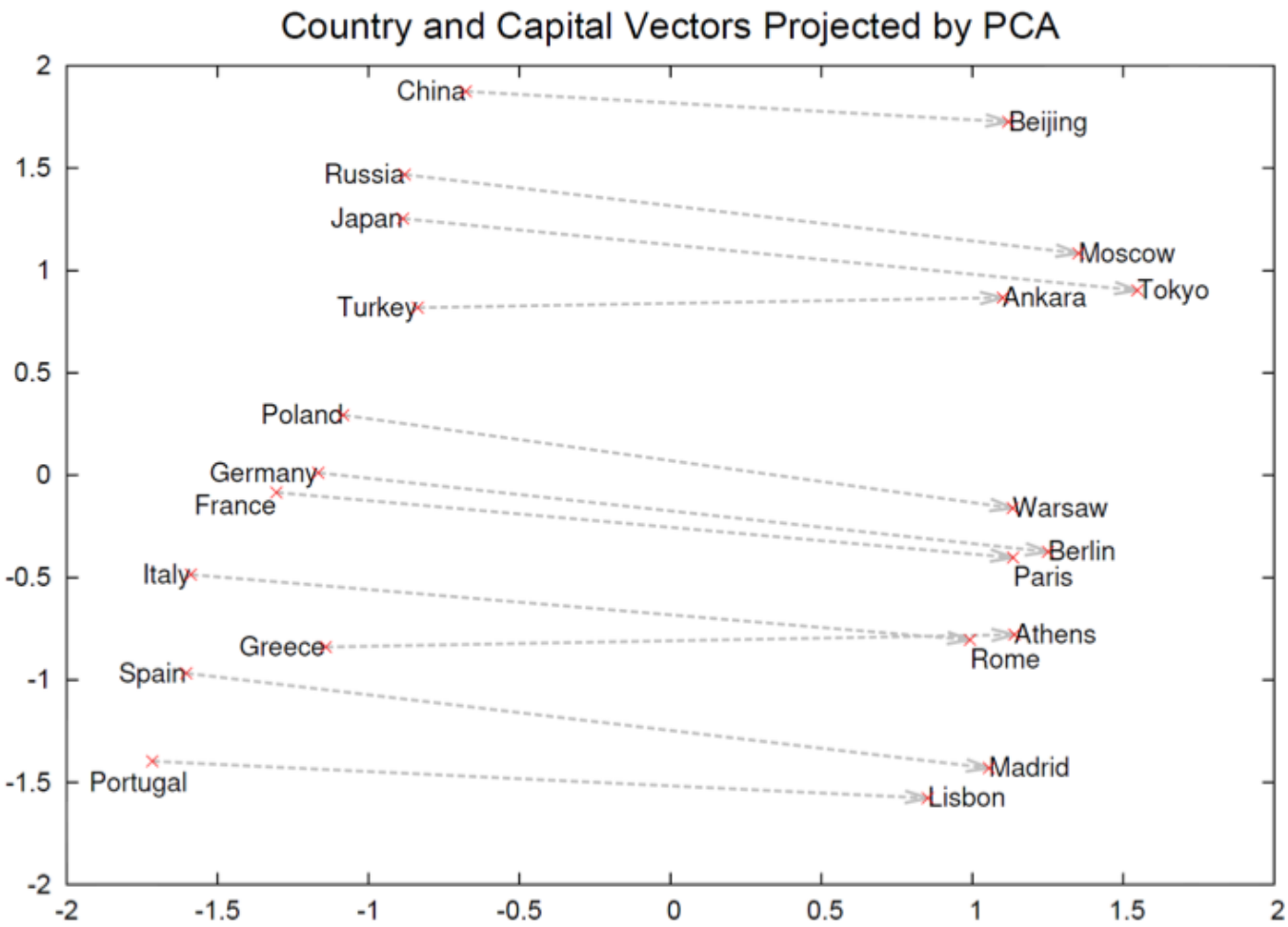
- High quality word vectors boost performance of all NLP tasks, including document classification, machine translation, information retrieval...
- Example for English to Spanish machine translation:



About 90% reported accuracy (Mikolov et al. 2013c)

Mikolov, T., Le, Q. V., & Sutskever, I. (2013). Exploiting similarities among languages for machine translation. arXiv:1309.4168.

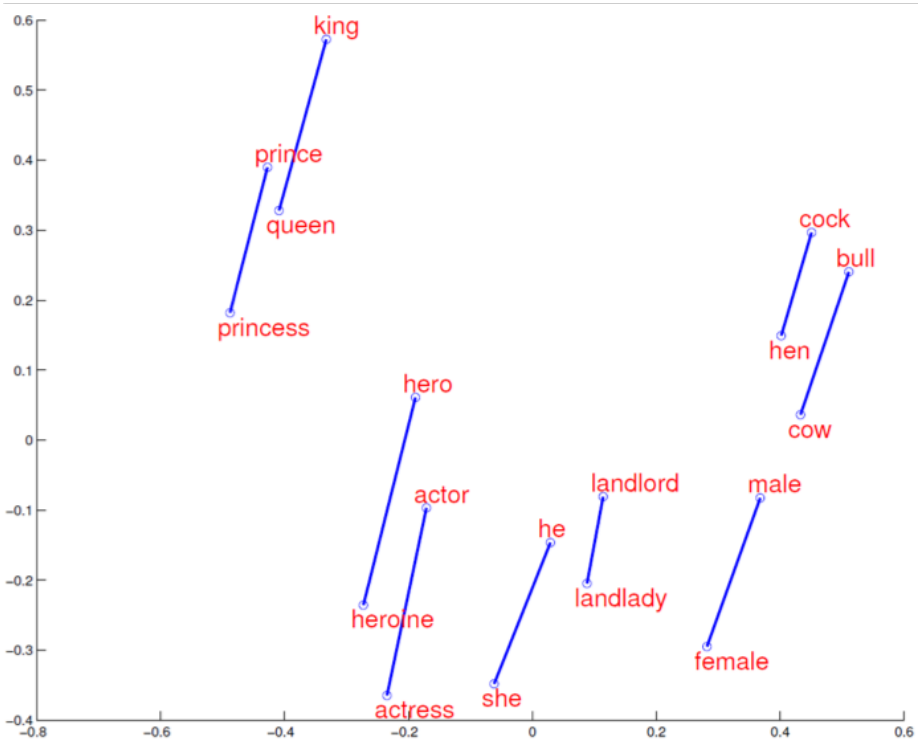
Remarkable properties of word vectors



regularities between words are encoded in difference vectors e.g., constant **country-capital** difference vector

Mikolov et al. (2013b) Distributed representations of words and phrases and their compositionality

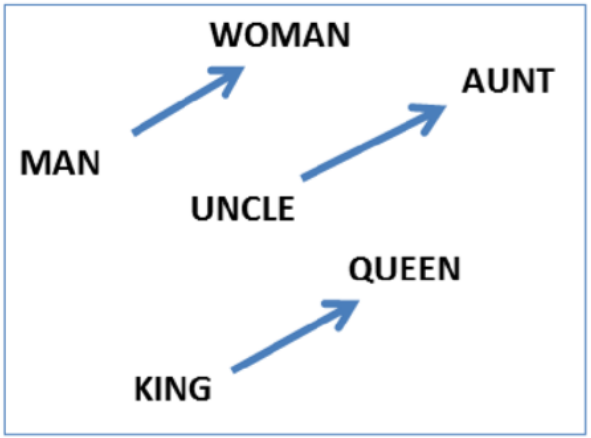
Remarkable properties of word vectors



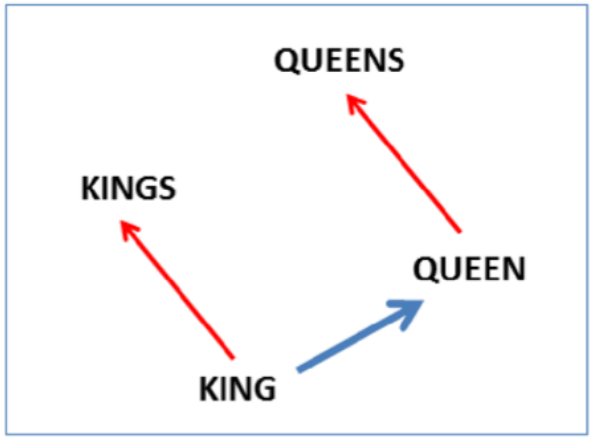
constant **female-male** difference vector

http://www.scribd.com/doc/285890694/NIPS-DeepLearningWorkshop-NNforText#scribd

Remarkable properties of word vectors



constant **female-male** difference vector



constant **singular-plural** difference vector

- Vector operations are supported and make intuitive sense:

$w_{king} - w_{man} + w_{woman} \cong w_{queen}$

$w_{paris} - w_{france} + w_{italy} \cong w_{rome}$

$w_{windows} - w_{microsoft} + w_{google} \cong w_{android}$

$w_{einstein} - w_{scientist} + w_{painter} \cong w_{picasso}$

$w_{his} - w_{he} + w_{she} \cong w_{her}$

$w_{cu} - w_{copper} + w_{gold} \cong w_{au}$

- Online demo

http://rare-technologies.com/word2vec-tutorial/

Outline

1. MLP applications to NLP
   - MLPs for Word embeddings

2. Recurrent NNs + LSTMs

Recurrent Neural Networks

- CNNs are good at image classification
  - Input: an image
  - Output: probability of the class
    - $p(\text{Red Panda} \mid \text{image}) = 0.9$
    - $p(\text{Cat} \mid \text{image}) = 0.1$



- Sequence learning: study of ML algorithms for sequential data (time series, language...)
  - Translate: "machine learning is a challenging topic" to French: "L'apprentissage automatique est un sujet difficile"
  - Predict next word: "John visited Paris, the capital of... "
  - Input and output strongly correlated within the sequence.

Sequences are everywhere, and order carries meaning

- Text, speech frames, sensor and financial series, video, DNA/protein, clickstreams.
- Shared properties:
  - variable length T;
  - **order matters**: *dog bites man* $\neq$ *man bites dog*;
  - long-range dependencies.
- Task shapes: classify a sequence, label each element, generate a sequence, map sequence $\rightarrow$ sequence.

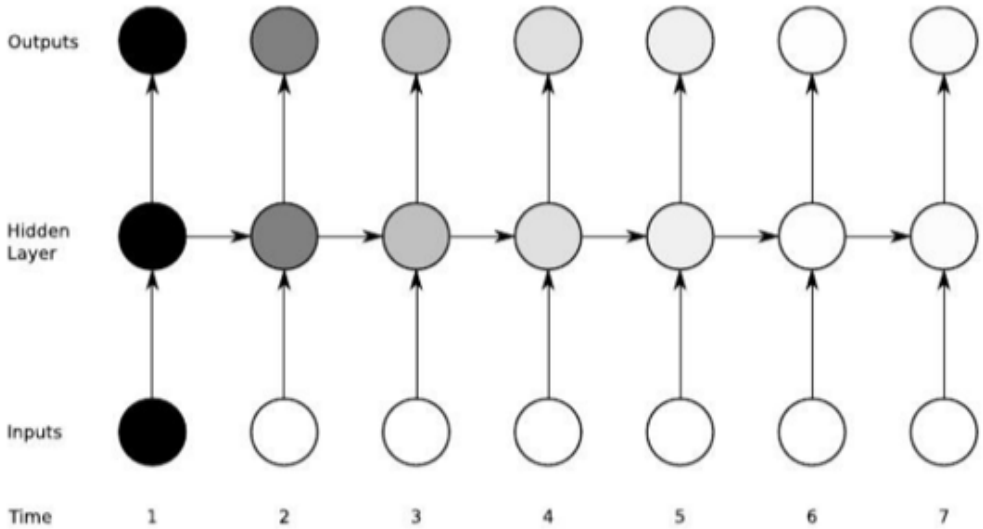
The **keys** to the cabinet in the hallway **are** on the table.

A bag-of-words model cannot tell *dog bites man* from *man bites dog*: it sees the same counts.

how many tokens separate *keys* and *are*, and what must the model carry across that gap?

Recurrent Neural Networks

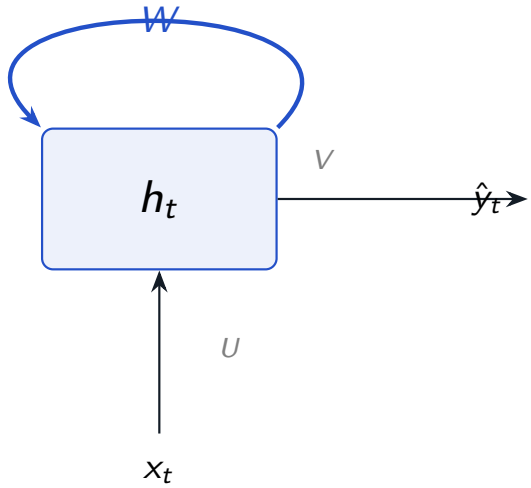
- Update the hidden state in a deterministic nonlinear way.
- RNNs are powerful.
- Distributed hidden state that allows storing past information efficiently.
- Non-linear dynamics for updating states in complicated ways.
- Pure deterministic updates; weight sharing.



The vanilla (Elman) RNN

$$h_t = \tanh\big(Wh_{t-1} + Ux_t + b_h\big)$$

$$\hat{y}_t = \mathrm{softmax}\big(Vh_t + b_y\big)$$

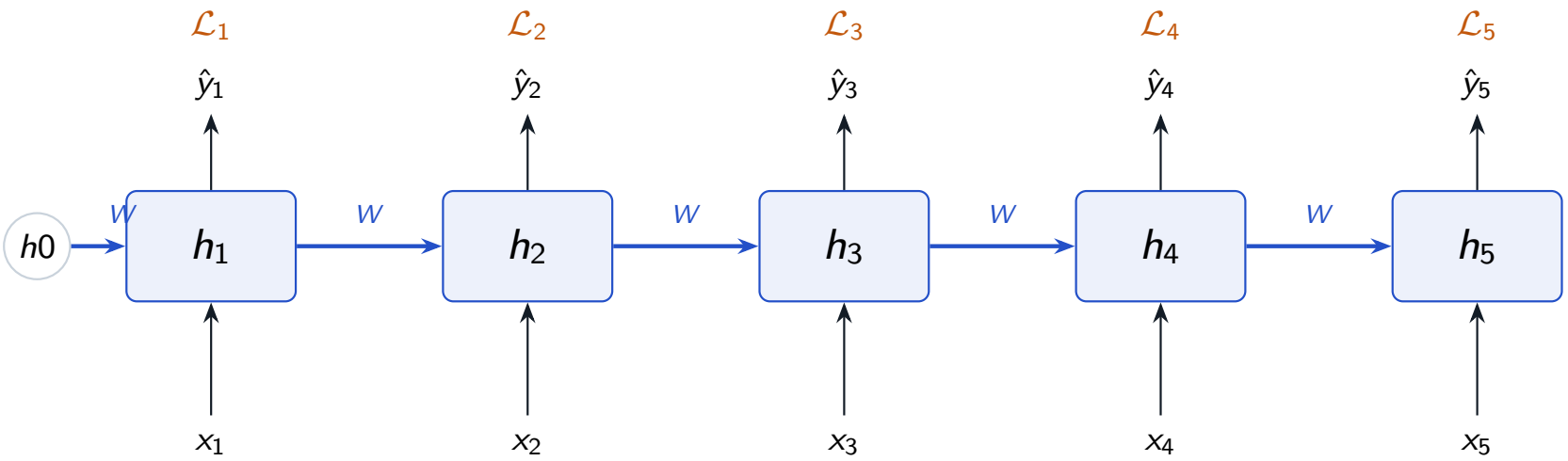


- h_t is a lossy, learned summary of $x_{1:t}$: the memory.
- The same W, U, V at **every** step.
- $h_0 = 0$ (or learned); a dynamical system $h_t = f_\theta(h_{t-1}, x_t)$.
- Parameters $d_h^2 + d_h d_x + d_h d_y + d_h + d_y$ are **independent of** T; activations stored for backprop are not $(T \cdot d_h)$.

Example: $d_x = 300$, $d_h = 512$, $d_y = 10\,000$
$\Rightarrow$ 5,545,984 parameters, whatever T is.

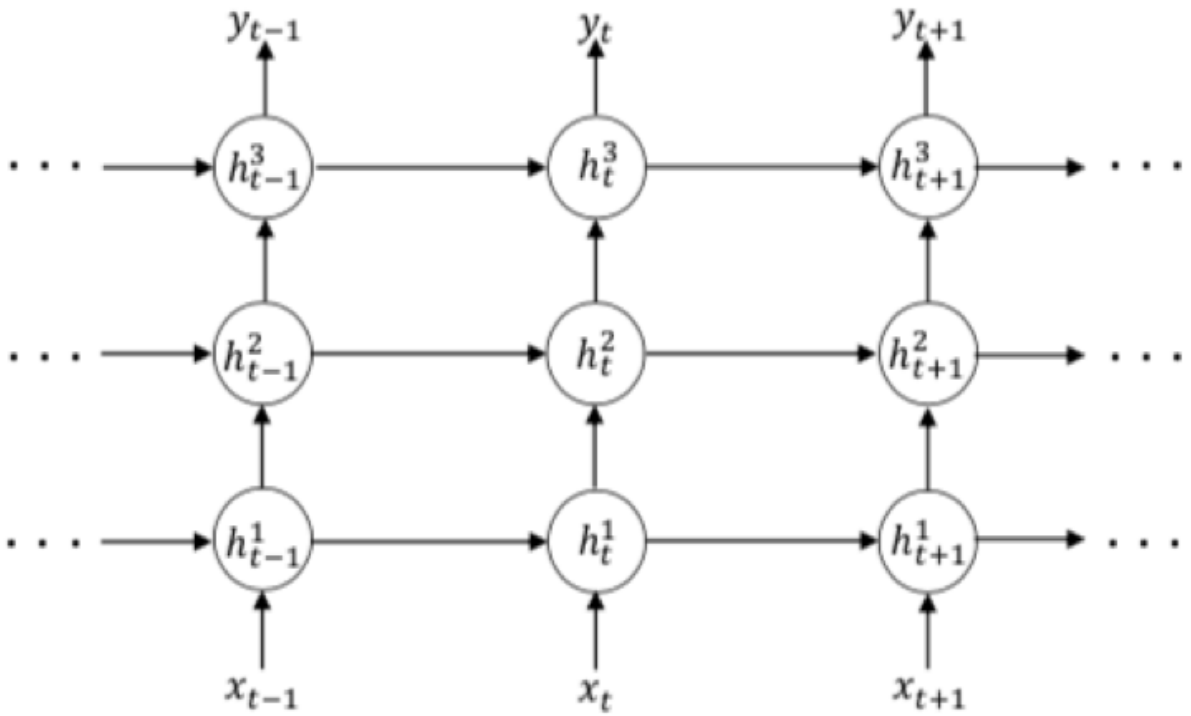
Elman (1990); Jordan (1986)

Unrolled, an RNN is a deep network with tied weights



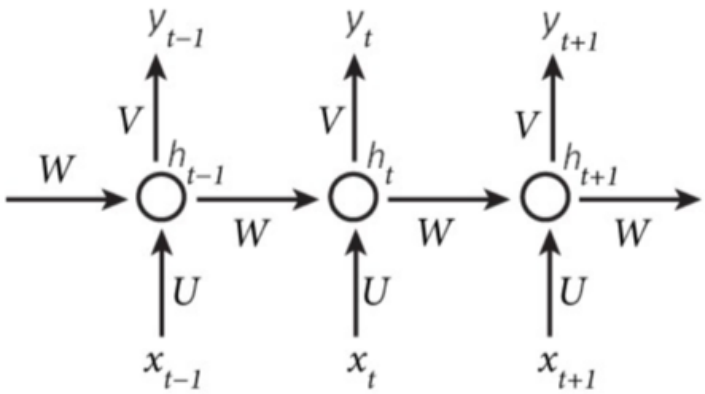
- Unrolling for T steps gives a feed-forward net of **depth** T.
- Total loss $\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t(\hat{y}_t, y_t)$.
- Every blue arrow is the *same* matrix: the gradient for W sums over all copies.
- The forward pass is **sequential**: h_t needs h_{t-1}.

Deep RNNs



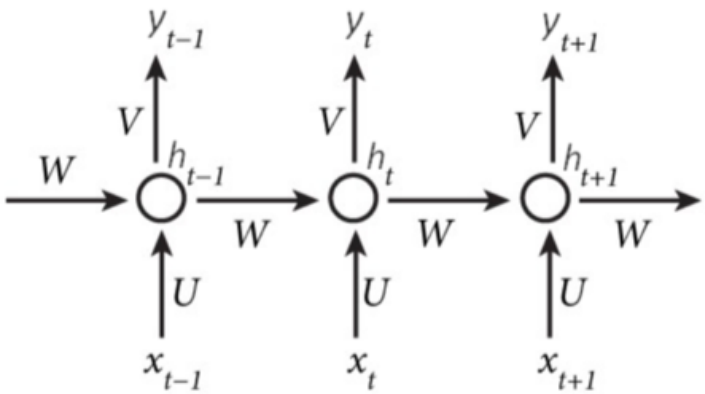
3 steps of an unrolled deep RNN. Hidden state of unit in inner layer serves as input to layer above.

Recurrent Neural Networks



- CNNs are naturally efficient with grids.
- RNNs were specifically developed to be used with sequences:
  - time series, words, sentences.
  - language modeling $P[w_n|w_1, \ldots, w_{n-1}]$.
  - generate new and quite convincing sentences from scratch.
  - chain of simple neural layers sharing the same parameters.

Learning in RNNs



- Assuming input $x_1, \ldots, x_T$
- At a single time step (omit biases):

$$h_t = \sigma(Wh_{t-1} + Ux_t)$$

$$y_t = \text{softmax}(Vh_t)$$

Learning in RNNs

$$h_t = \sigma(Wh_{t-1} + Ux_t)$$
$$y_t = \text{softmax}(Vh_t)$$

- Main idea: we use the same set of W, U, V weights at all time steps!
- $h_0 \in \mathbb{R}^{H \times H}$ initialization vector for hidden layer at $t = 0$
- $\hat{y} \in \mathbb{R}^{|V|}$ is a probability distribution over the vocabulary
- Loss function (@ time t): cross entropy

$$J^{(t)}(\theta) = -\sum_{j=1}^{|V|} y_{t,j} \log \hat{y}_{t,j}$$

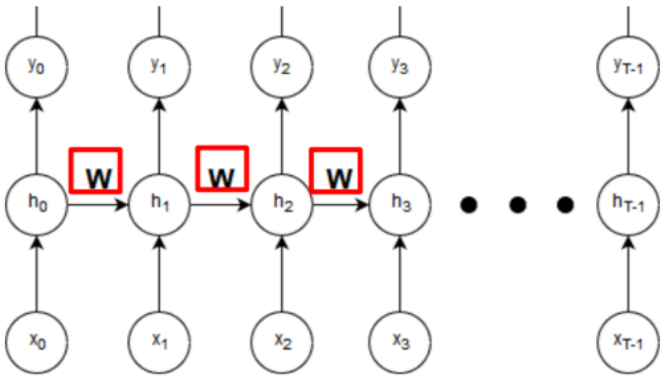
Learning in RNNs - backpropagation in time

- Learning the weights of $\mathbf{W}$:

$$\mathbf{W} \rightarrow \mathbf{W} - \alpha \frac{\partial \mathbf{y}}{\partial \mathbf{W}}$$

- Involves accumulation of gradients from each output $\mathbf{y}_j$:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{W}} = \sum_{j=0}^{T-1} \frac{\partial \mathbf{y}_j}{\partial \mathbf{W}}$$



- Temporal chain backward:

$$\frac{\partial y_j}{\partial \mathbf{W}} = \sum_{k=1}^{j} \frac{\partial y_j}{\partial h_k} \frac{\partial h_k}{\partial \mathbf{W}}$$

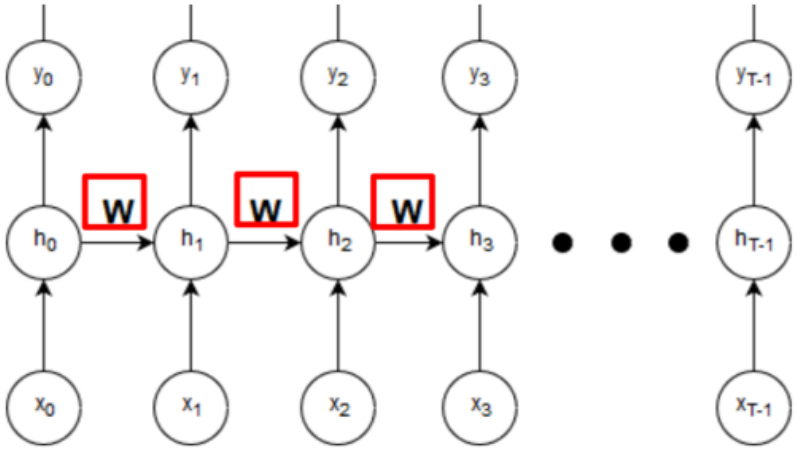
Learning in RNNs - backpropagation in time

- Expanding via intermediate hidden states:

$$\frac{\partial y_j}{\partial \mathbf{W}} = \sum_{k=1}^{j} \frac{\partial y_j}{\partial h_j} \frac{\partial h_j}{\partial h_k} \frac{\partial h_k}{\partial \mathbf{W}}$$

- By the chain rule:

$$\frac{\partial h_j}{\partial h_k} = \prod_{m=k+1}^{j} \frac{\partial h_m}{\partial h_{m-1}}$$

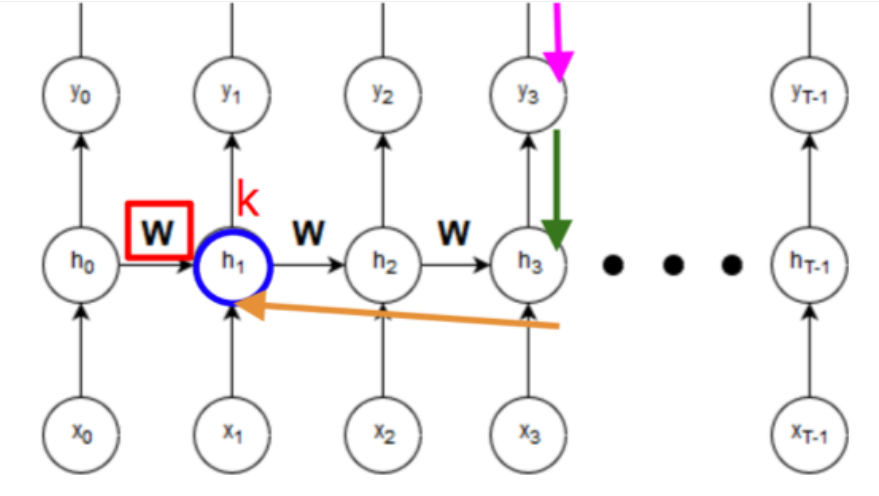


Learning in RNNs - back propagation in time

Backprop on the unrolled graph:

$$\frac{\partial y_j}{\partial \mathbf{W}} = \sum_{j=0}^{T-1} \sum_{k=1}^{j} \frac{\partial y_j}{\partial h_j} \left(\prod_{m=k+1}^{j} \frac{\partial h_m}{\partial h_{m-1}} \right) \frac{\partial h_k}{\partial \mathbf{W}}$$

- Cost: $O(T)$ time and $O(T)$ memory.



Learning in RNNs - vanishing/exploding gradients

$$\frac{\partial y_j}{\partial \mathbf{W}} = \sum_{j=0}^{T-1} \sum_{k=1}^{j} \frac{\partial y_j}{\partial h_j} \left(\prod_{m=k+1}^{j} \frac{\partial h_m}{\partial h_{m-1}} \right) \frac{\partial h_k}{\partial \mathbf{W}}$$

$$h_m = f(\mathbf{W}_h h_{m-1} + \mathbf{W}_x x_m)$$

$$\frac{\partial h_m}{\partial h_{m-1}} = \mathbf{W}_h^T \operatorname{diag}\left(f'(\mathbf{W}_h h_{m-1} + \mathbf{W}_x x_m)\right)$$

$$\frac{\partial h_j}{\partial h_k} = \prod_{m=k+1}^{j} \mathbf{W}_h^T \operatorname{diag}\left(f'(\mathbf{W}_h h_{m-1} + \mathbf{W}_x x_m)\right)$$

Weight Matrix

Derivative of activation function

Repeated matrix multiplications leads to vanishing and exploding gradients

The Jacobian product in RNNs

With preactivation:

$$a_j = W h_{j-1} + U x_j + b_h, \quad h_j = \tanh(a_j)$$

Differentiating current hidden state h_j with respect to h_{j-1}:

$$\frac{\partial h_j}{\partial h_{j-1}} = \text{diag}\left(\tanh'(a_j)\right) W$$

Backpropagating error from step t back to step k:

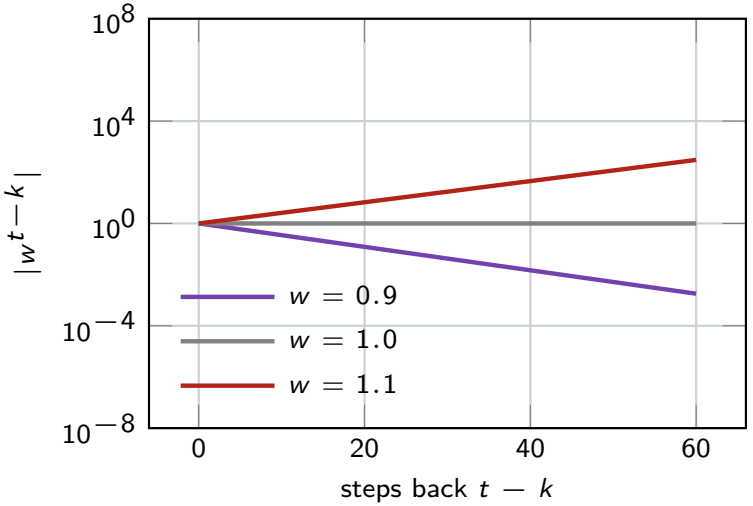
$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}}$$

Bengio, Simard & Frasconi (1994); Pascanu, Mikolov & Bengio (2013)

$$\left\| \prod_{j=k+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \right\| \leq \left(\gamma\, \sigma_{\max}(W)\right)^{t-k}$$

- $\gamma = \sup |\tanh'| = 1$ (1/4 for sigmoid).
- $\sigma_{\max} < 1/\gamma \Rightarrow$ gradients vanish exponentially
- $\sigma_{\max} > 1/\gamma \Rightarrow$ gradients may explode

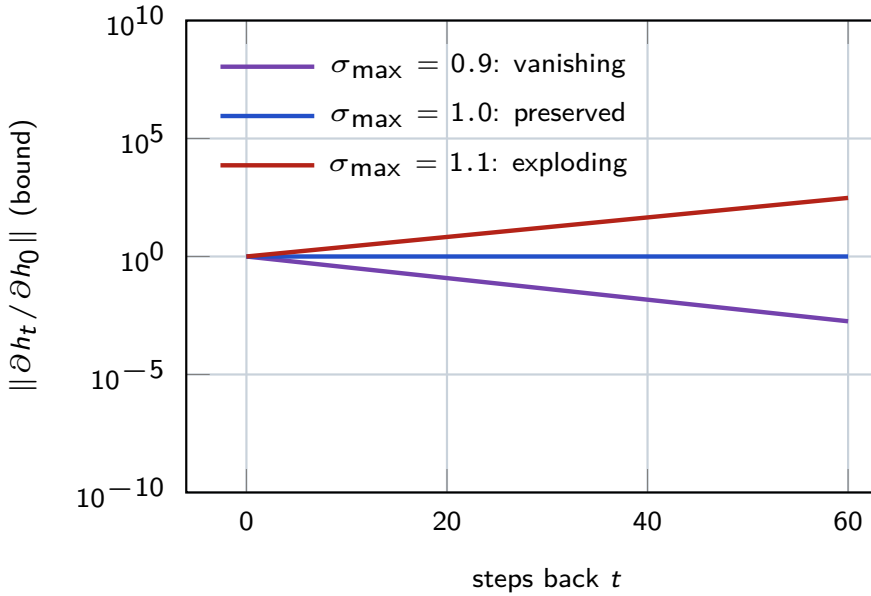
Scalar intuition: $h_t = w\, h_{t-1} \Rightarrow \partial h_t / \partial h_k = w^{t-k}$



$0.9^{50} \approx 0.005 \quad 1.1^{50} \approx 117$

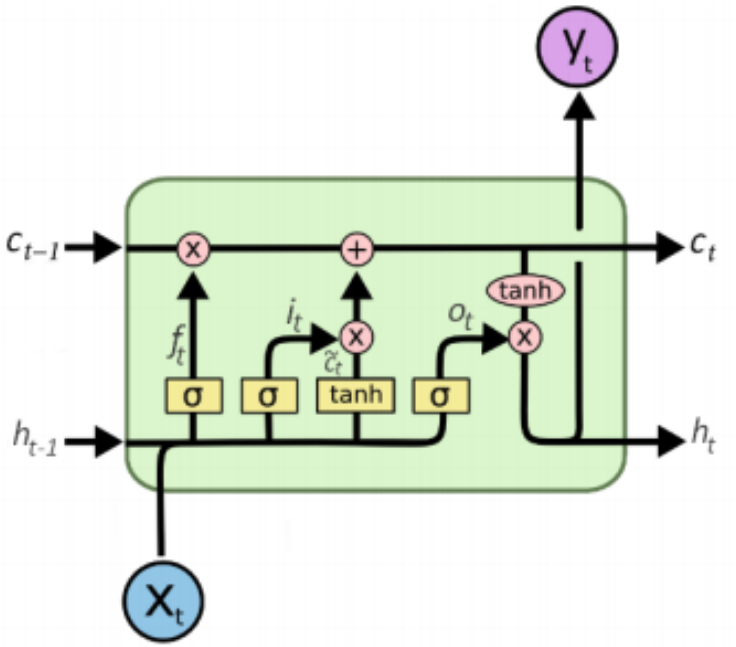
Vanishing vs. exploding: what actually goes wrong

- **Vanishing:** training is stable, but the model **cannot learn long-range dependencies**. Short-range terms dominate the sum, and nothing visibly breaks – a silent failure.
- **Exploding:** loss spikes, NaNs, easy to detect.
- Saturated tanh units ($\tanh' \approx 0$) make vanishing worse.



Measured products of true Jacobians sit *below* these bounds due to saturation.

LSTM units

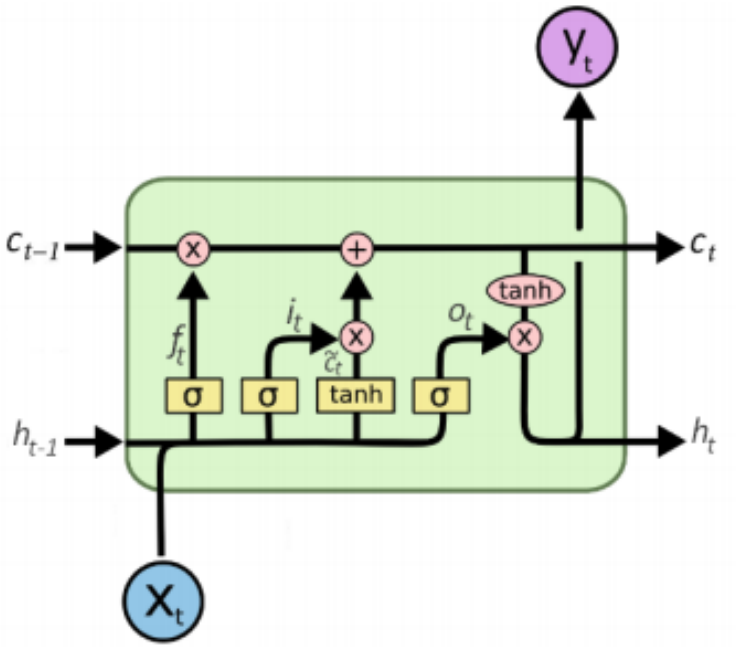


The LSTM unit. Adapted from Chris Colah's blog

1. forget gate layer: $f_t = \sigma(U_f x_t + W_f h_{t-1} + b_f)$
2. input gate layer: $i_t = \sigma(U_i x_t + W_i h_{t-1} + b_i)$
3. candidate values computation: $\tilde{c}_t = \tanh(U_c x_t + W_c h_{t-1} + b_c)$
4. output gate layer: $o_t = \sigma(U_o x_t + W_o h_{t-1} + b_o)$

- Tackles vanishing gradients and tracks long-term dependencies.
- **Cell state** (C_t): "memory" carrying context across time steps.
- **Hidden state** (h_t): short-term output passed to next unit.
- **Gates**:
  - **Forget gate:** what to erase from memory.
  - **Input gate:** new information to store.
  - **Output gate:** output at current step.

LSTM units



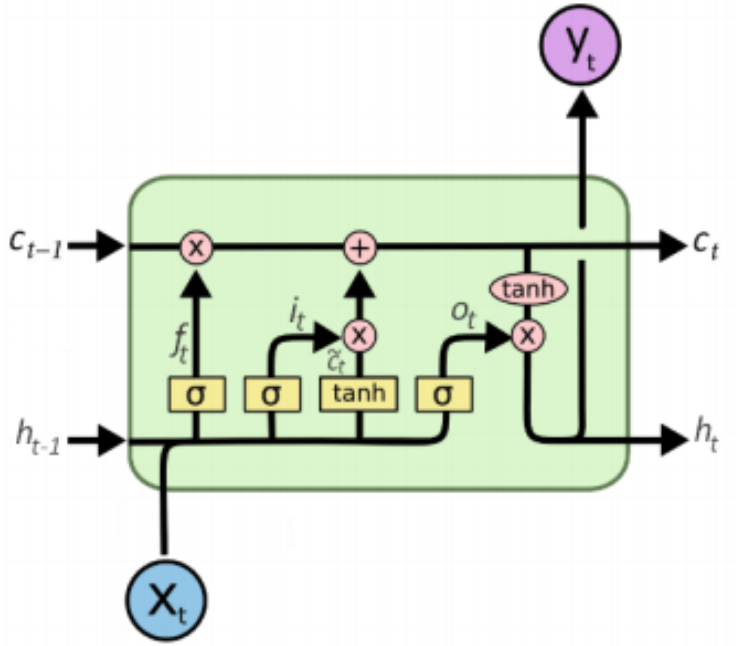
The LSTM unit. Adapted from Chris Colah's blog

1. forget gate layer: $f_t = \sigma(U_f x_t + W_f h_{t-1} + b_f)$
2. input gate layer: $i_t = \sigma(U_i x_t + W_i h_{t-1} + b_i)$
3. candidate computation: $\tilde{c}_t = \tanh(U_c x_t + W_c h_{t-1} + b_c)$
4. output gate layer: $o_t = \sigma(U_o x_t + W_o h_{t-1} + b_o)$

Given input x_t and hidden state h_{t-1}:

- f_t determines how much of cell state c_{t-1} is forgotten.
- i_t determines how much of candidate $\tilde{c}_t$ is written to memory:
  - $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$
  - $h_t = \tanh(c_t) \odot o_t$
  - $y_t = \text{softmax}(Vh_t)$

LSTM units



The LSTM unit. Adapted from Chris Colah's blog

1. forget gate: $f_t = \sigma(U_f x_t + W_f h_{t-1} + b_f)$
2. input gate: $i_t = \sigma(U_i x_t + W_i h_{t-1} + b_i)$
3. candidate: $\tilde{c}_t = \tanh(U_c x_t + W_c h_{t-1} + b_c)$
4. output gate: $o_t = \sigma(U_o x_t + W_o h_{t-1} + b_o)$

Example: Predicting the Next Word
Assume training an LSTM on:

"The weather is very ____"

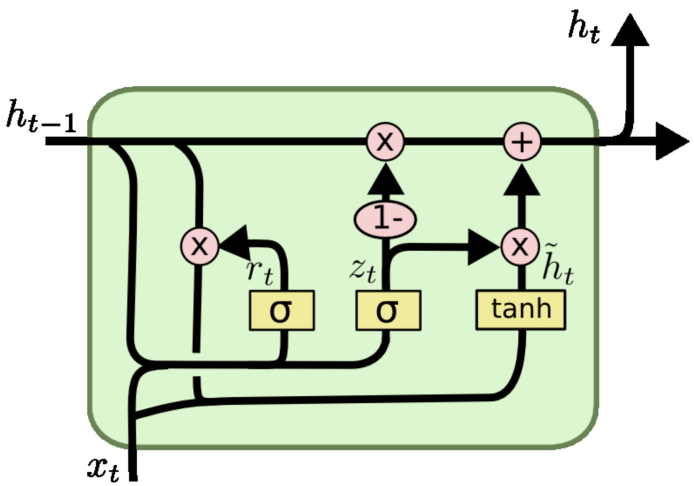
The LSTM processes:

$x_1 =$ "The", $x_2 =$ "weather", $x_3 =$ "is",

At each step, memory is updated. When it reaches the blank, it predicts:

$P($"hot"$) = 0.6$, $P($"cold"$) = 0.3$, $P($"rain

GRU units



The GRU unit

Gated Recurrent Unit (GRU): simplified version of LSTM [Cho et al. (2014)]. Designed to mitigate vanishing gradients with fewer parameters.

Combines cell and hidden state into h_t using two gates:

- **Update Gate** (z_t): how much past information to keep.
- **Reset Gate** (r_t): how much past information to discard.

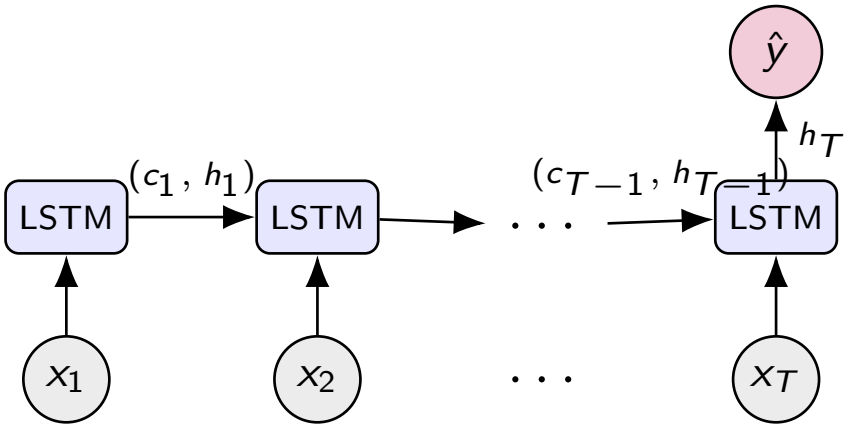
GRU Updates:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$
$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h)$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Application Example: Sentiment Classification

Task: Map a sequence of input tokens $x_{1:T}$ to a target polarity class $\hat{y} \in \{0, 1\}$.

Pipeline Flow



Mathematical Formulation

- **Token Encoding:** $x_t = \mathbf{E}\, w_t \quad (x_t \in \mathbb{R}^d)$
- **Recurrent Step:** $(c_t, h_t) = \text{LSTM}(x_t, c_{t-1}, h_{t-1})$
- **Classifier Head:** $\hat{y} = \sigma(\mathbf{W}_y h_T + b_y)$

Why LSTM?

- Retains early signals (e.g., *"Not at all satisfied. . . "*) via linear updates c_t.
- Suppresses vanishing gradients across long sequences via additive error flow.

GRU vs LSTM units

Feature	LSTM	GRU
Number of gates	3 (input, forget, output)	2 (update, reset)
Cell state	Separate cell C_t	Combined with hidden h_t
Parameters	More (slower to train)	Fewer (faster to train)
Performance	Often slightly better on complex data	Comparable or better on smaller datasets

Context & Senses

- "A word is defined by *the company it keeps*" (Firth, 1957)
- Words are *ambiguous*:
  - The amount of deposits in the banks decreased by 3.5% in March
  - The trees on bank of the river were offering their shade to the visitors
- We need different vectors to represent all word/token senses

Deep Contextualized Word Representations

Best paper NAACL 2018

Transfer learning has been used in vision since 2012 (ImageNet); it is used in NLP since 2018!



ELMo: Embeddings from Language Models

Initial problem: traditional word vectors map each word to a single, context-independent vector. But some words have more than one sense! (polysemy)

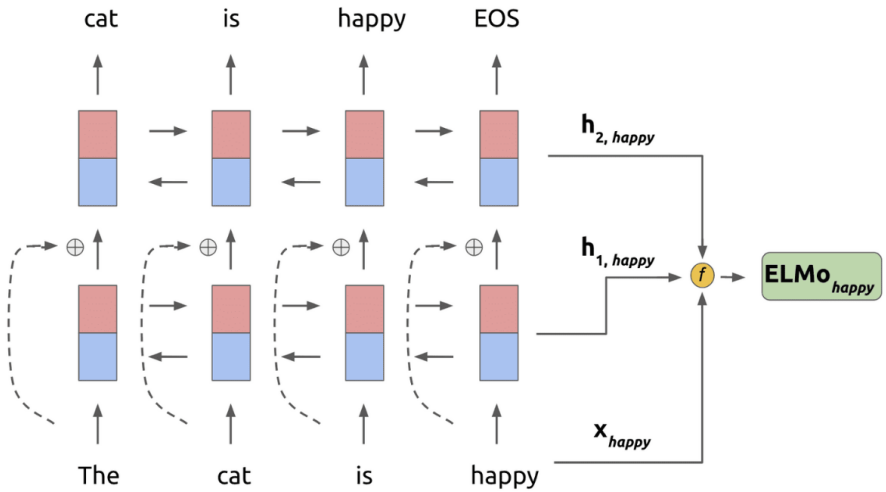
- e.g., *bank, get, wood, play, mean...*

Solution:

- each token is assigned a representation based on the entire input sentence
- use the internal representations of an RNN language model pre-trained on a large dataset

Deep Contextualized Word Representations

- A bidirectional LSTM is trained with a coupled language model (LM) on a large text corpus — ELMo (Embeddings from Language Models) representations.



- Forward LM: $p(t_1, t_2, \ldots, t_N) = \prod_{k=1}^{N} p(t_k \mid t_1, t_2, \ldots, t_{k-1})$

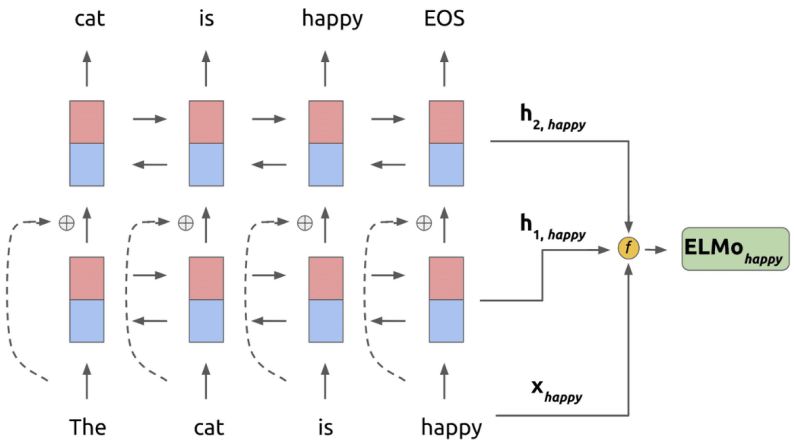
- Backward LM: $p(t_1, t_2, \ldots, t_N) = \prod_{k=1}^{N} p(t_k \mid t_{k+1}, t_{k+2}, \ldots, t_N)$

Graphic from: https://www.mihaileric.com/posts/deep-contextualized-word-representations-elmo/

Deep Contextualized Word Representations

ELMo representations are **deep**: a function of all of the internal layers of the biLM.

- learn a linear combination of the vectors stacked above each input word for each end task;
- combining the internal states in this manner allows for very rich word representations;
- higher-level LSTM states capture context-dependent aspects of word meaning (e.g., they can be used without modification to perform well on supervised word sense disambiguation tasks);
- lower-level states model aspects of syntax (e.g., they can be used to do part-of-speech tagging);
- simultaneously exposing all of these signals is highly beneficial, allowing the learned models to select the types of semi-supervision that are most useful for each end task.

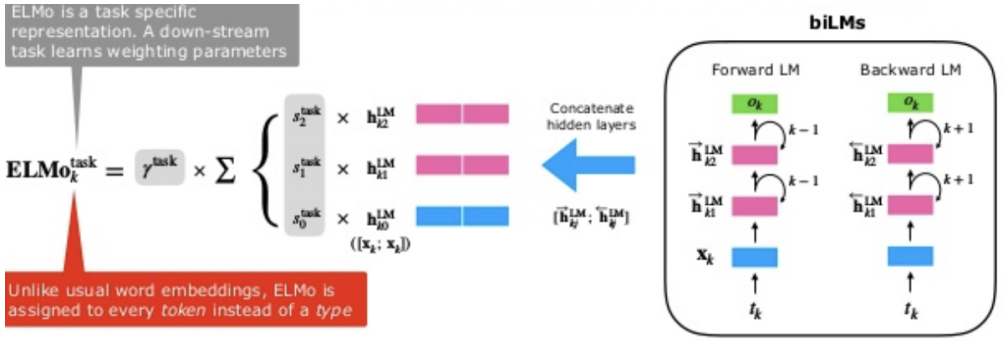


Deep Contextualized Word Representations

Semi-supervised approach:

1. a deep bidirectional RNN language model is pretrained on a large dataset;
2. the vector of each word in a given input sentence is computed as a weighted sum of the RNN hidden states.

(1) is *unsupervised*; (2) weights are learned in a *supervised* way on some task-specific dataset.



$$\text{ELMo}_k^{task} = \gamma^{task} \sum_{j=0}^{L} s_j^{task} \, \mathbf{h}_{k,j}^{LM}$$

$\mathbf{h}_{k,j}^{LM}$ is the k-th hidden representation of the j-th layer of the biRNN LM; s^{task} is the softmax weight vector; γ^{task} is a scaling parameter (for optimisation). The authors use $L = 2$ in all experiments.

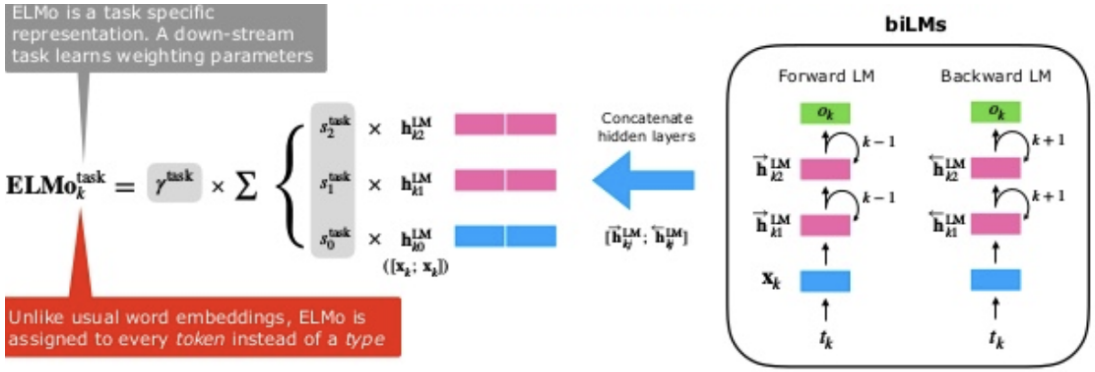
Graphic from: https://www.mihaileric.com/posts/deep-contextualized-word-representations-elmo/

Deep Contextualized Word Representations

Use of ELMo in practice, on a task-specific dataset:

1. use the pretrained language model in prediction mode and store $\mathbf{h}_{k,j}^{LM}$ for each word (each k) and each layer (each j);
2. concatenate ELMo_k^{task} with the corresponding input vector* of whatever supervised model is used to solve the task (e.g., RNN, CNN, feed-forward...);
3. update s^{task} and γ^{task} with the other parameters of the supervised model during training.

*given by word2vec, GloVe, etc.



Results: ELMo improves over the baselines on 8 tasks, ranging from question answering to co-reference resolution, sentiment analysis, POS tagging, and disambiguation.

Deep Contextualized Word Representations

Results:

TASK	PREVIOUS SOTA		OUR BASELINE	ELMO + BASELINE	INCREASE (ABSOLUTE/ RELATIVE)
SQuAD	Liu et al. (2017)	84.4	81.1	85.8	4.7 / 24.9%
SNLI	Chen et al. (2017)	88.6	88.0	88.7 ± 0.17	0.7 / 5.8%
SRL	He et al. (2017)	81.7	81.4	84.6	3.2 / 17.2%
Coref	Lee et al. (2017)	67.2	67.2	70.4	3.2 / 9.8%
NER	Peters et al. (2017)	91.93 ± 0.19	90.15	92.22 ± 0.10	2.06 / 21%
SST-5	McCann et al. (2017)	53.7	51.4	54.7 ± 0.5	3.3 / 6.8%

Table 1: Test set comparison of ELMo enhanced neural models with state-of-the-art single model baselines across six benchmark NLP tasks. The performance metric varies across tasks – accuracy for SNLI and SST-5; F_1 for SQuAD, SRL and NER; average F_1 for Coref. Due to the small test sizes for NER and SST-5, we report the mean and standard deviation across five runs with different random seeds. The "increase" column lists both the absolute and relative improvements over our baseline.

SQuAD: question answering; SNLI: entailment; SRL: semantic role labelling ("who did what to whom"); NER: named entity recognition; SST: sentiment analysis; Coref: clustering mentions in text that refer to the same underlying real-world entities.

Deep Contextualized Word Representations

Model	F_1
WordNet 1st Sense Baseline	65.9
Raganato et al. (2017a)	69.9
Iacobacci et al. (2016)	**70.1**
CoVe, First Layer	59.4
CoVe, Second Layer	*64.7*
biLM, First layer	67.4
biLM, Second layer	*69.0*

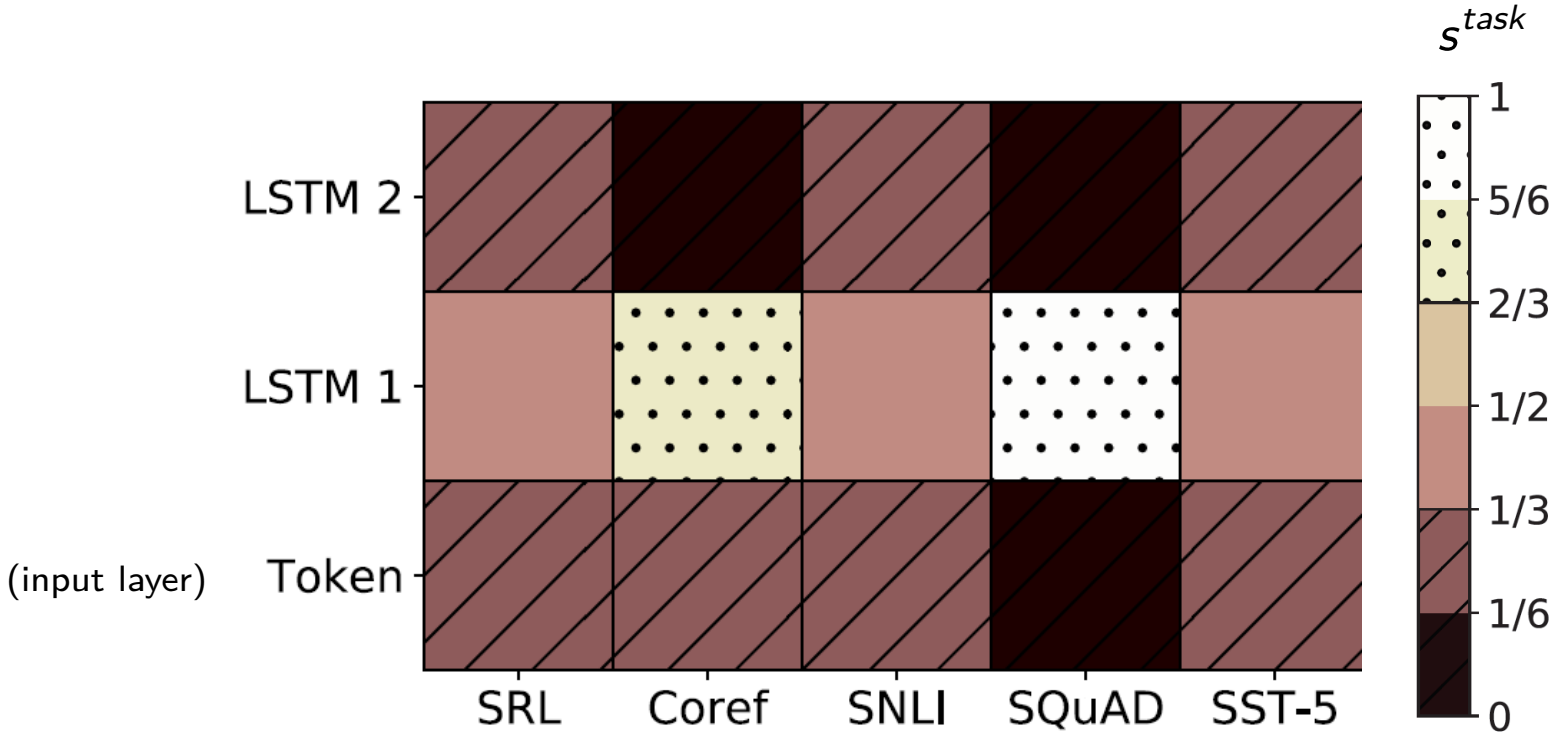
WSD

Model	Acc.
Collobert et al. (2011)	97.3
Ma and Hovy (2016)	97.6
Ling et al. (2015)	**97.8**
CoVe, First Layer	*93.3*
CoVe, Second Layer	92.8
biLM, First Layer	*97.3*
biLM, Second Layer	96.8

POS tagging

Deep Contextualized Word Representations

Not all layers are equally important for all tasks!



- The 1st layer clearly dominates for co-reference resolution and question answering.
- For the other tasks, the weights are more evenly distributed among layers.

Attention is ubiquitous in DL today

Image captioning



A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.

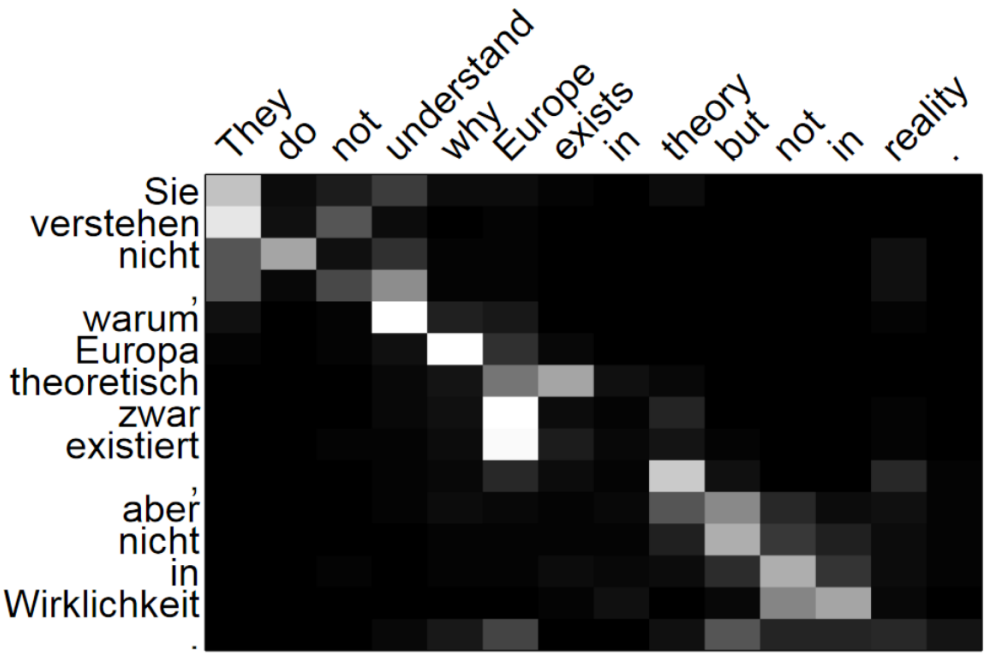


A giraffe standing in a forest with trees in the background.

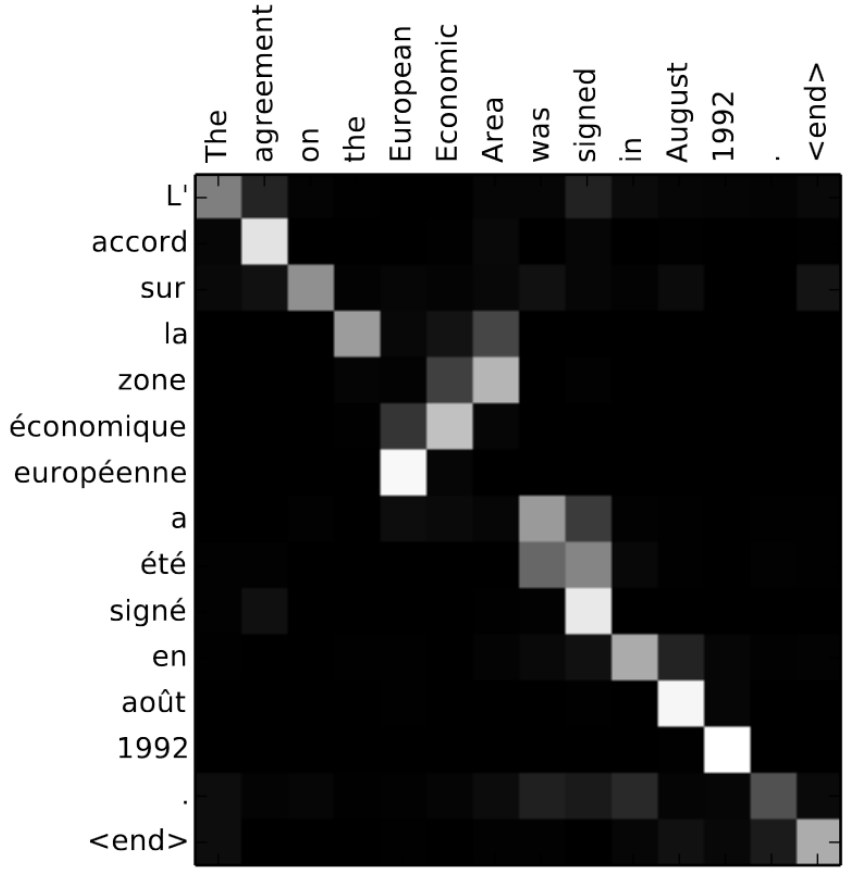
Show, attend and tell: Neural image caption generation with visual attention (Xu et al. 2015)

Attention is ubiquitous in DL today

Neural Machine Translation



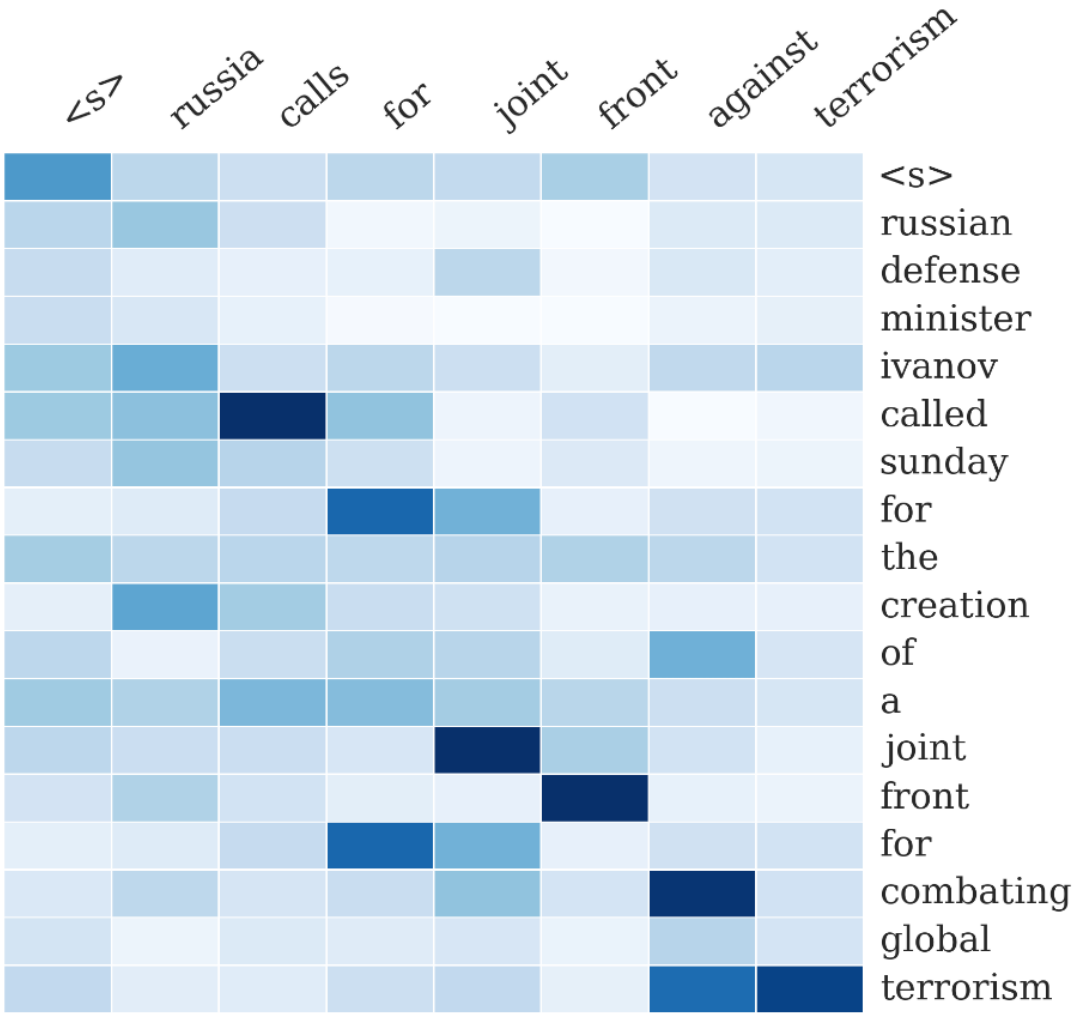
Effective approaches to attention-based neural machine translation (Luong et al. 2015)



Neural machine translation by jointly learning to align and translate (Bahdanau et al. 2014)

Attention is ubiquitous in DL today

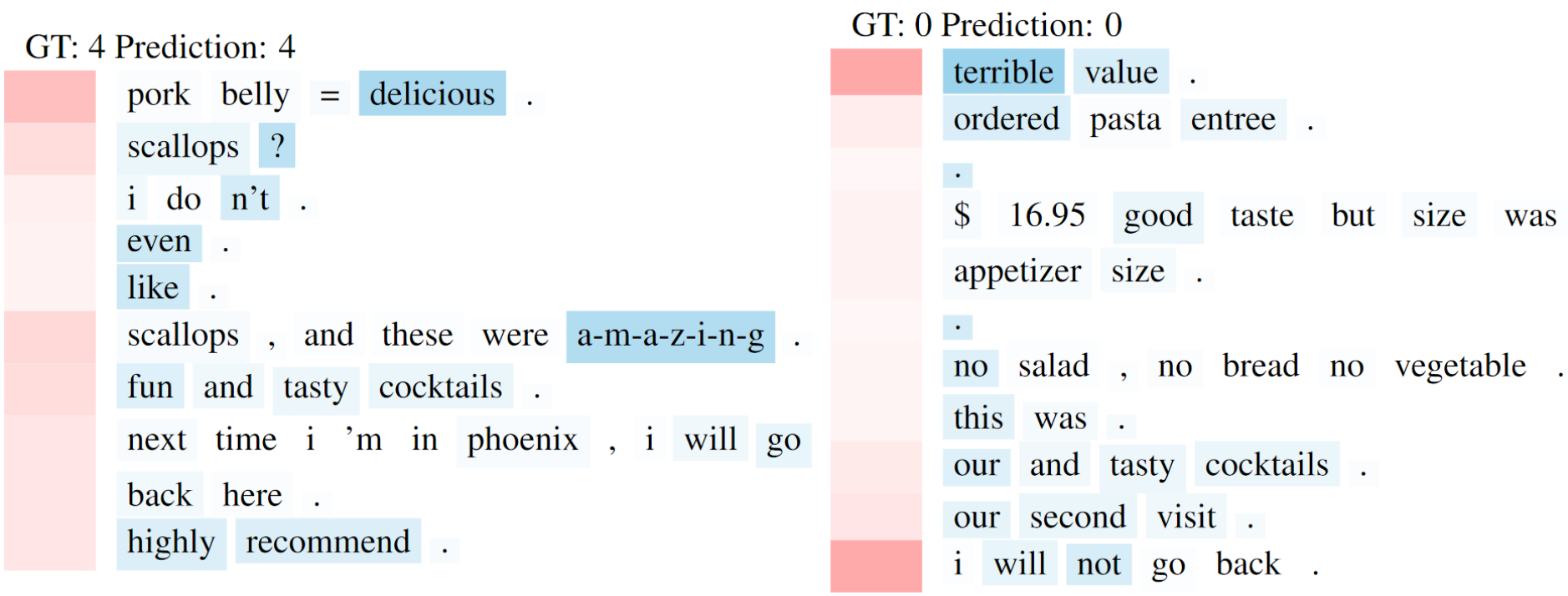
Abstractive summarization



A neural attention model for abstractive sentence summarization (Rush et al. 2015)

Attention is ubiquitous in DL today

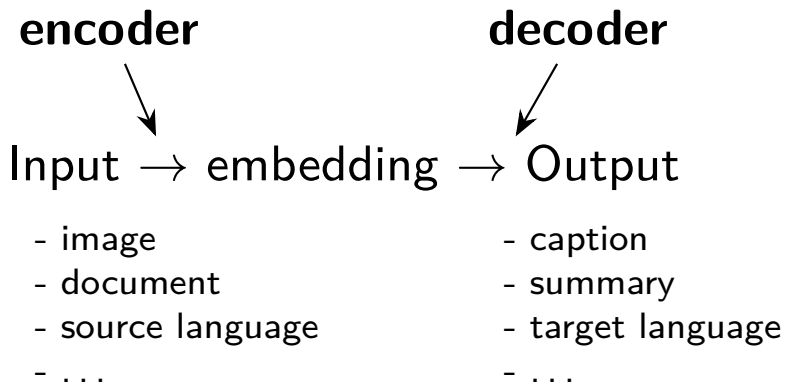
Sentiment analysis



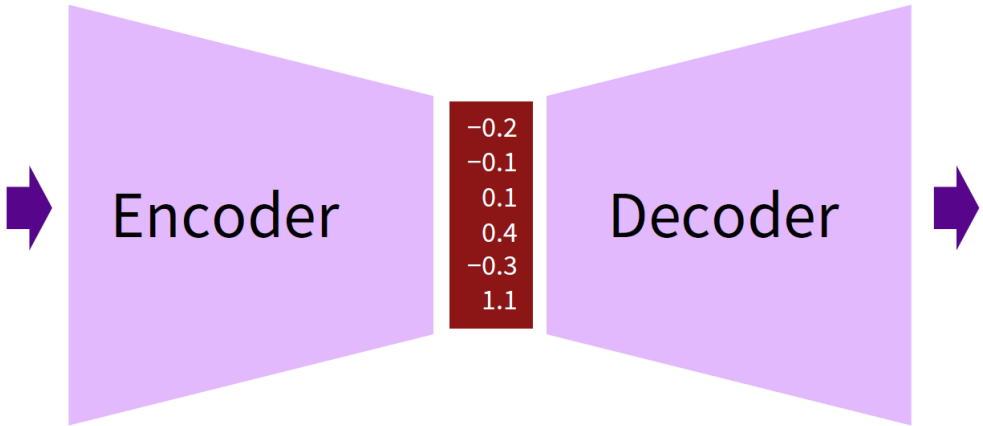
Hierarchical attention networks for document classification (Yang et al. 2016)

Encoder-Decoder Architectures

General idea:



- Known as *sequence-to-sequence* (seq2seq) when input and output are sequences (e.g., NLP applications)
- **The full architecture is differentiable ⇒ end-to-end training**



What is attention?

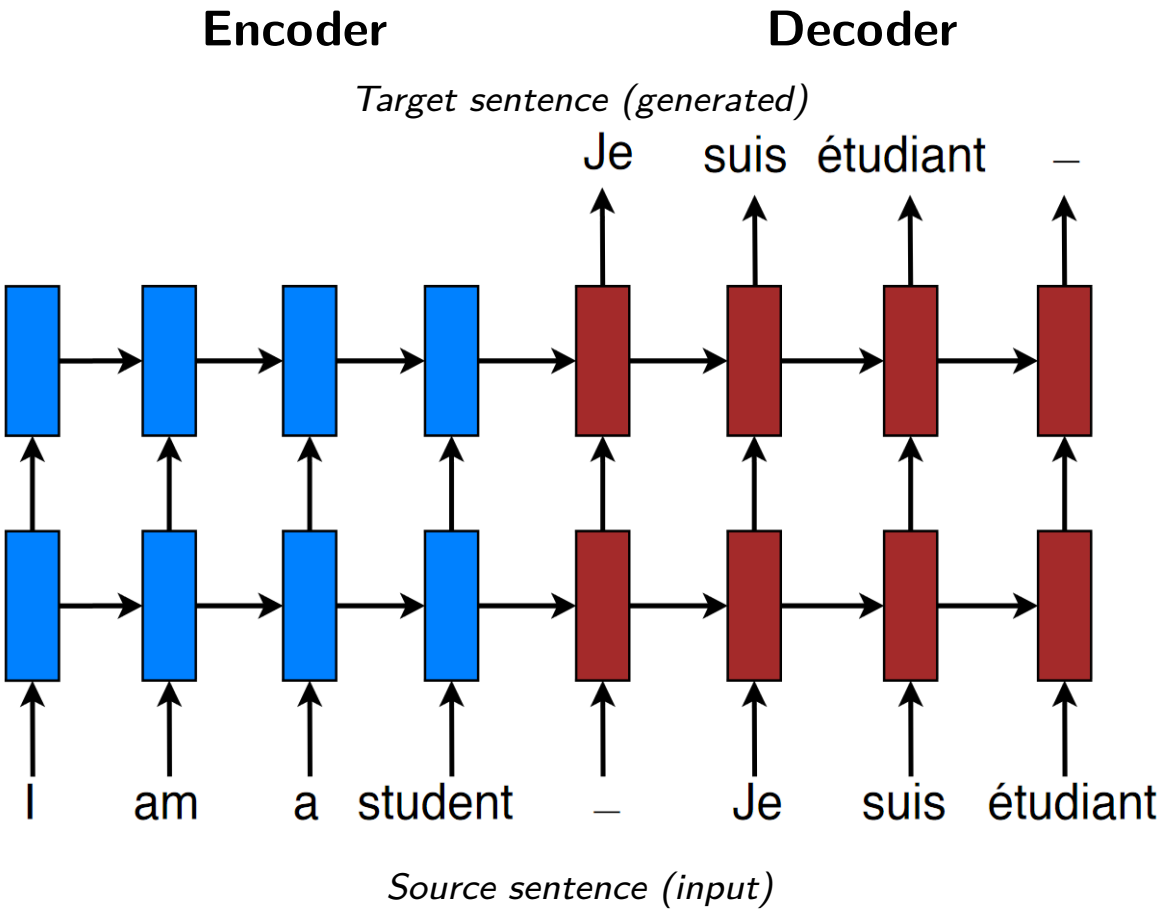
Objective

- Traditional models (e.g., the encoder) have to embed the input into a single fixed-length vector (lossy).
- Alternatively, information can be kept and stored into multiple vectors.
- This information can be retrieved later on (e.g., by the decoder) (Bahdanau et al. 2014).

Quick history:

- developed in the context of **encoder-decoder** architectures for neural machine translation (Bahdanau et al. 2014);
- rapidly applied to naturally related tasks like image captioning (Xu et al. 2015) and summarization (Luong et al. 2015);
- also proposed for encoders only, e.g. for text classification (Yang et al. 2015) and representation learning (Conneau et al. 2017). Known as *self or inner attention* in such cases.

Encoder-Decoder for Neural Machine Translation





Minh-Thang Luong

Research Scientist at Google
Verified email at google.com - Homepage
Deep Learning Natural Language Processing



Hieu Pham

Carnegie Mellon University, Google Brain
Verified email at google.com
Machine Learning

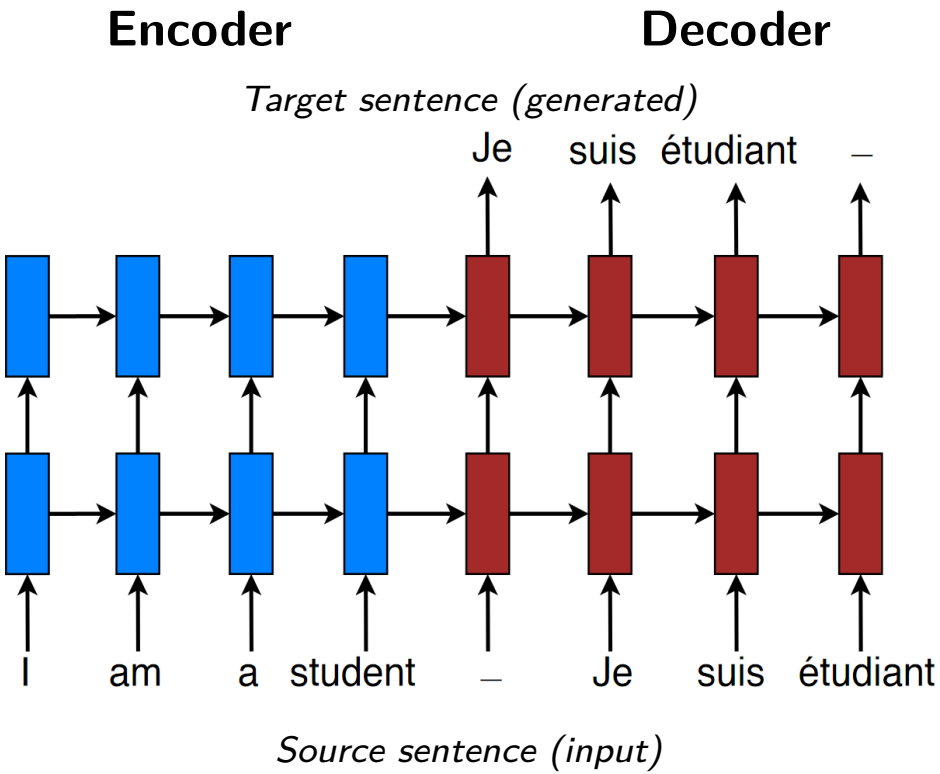


Christopher D Manning

Professor of Computer Science and Linguistics
Verified email at stanford.edu - Homepage
Natural Language Processing

Effective Approaches to Attention-Based Neural Machine Translation (2015)

Encoder-Decoder for Neural Machine Translation



- source: $(x_1, \ldots, x_{T_x})$
- target: $(y_1, \ldots, y_{T_y})$
- Both encoder & decoder are unidirectional deep RNNs (a.k.a. *stacking* RNNs)
- Training objective:

$$\operatorname*{argmax}_{\theta} \sum_{(x,y)\in \text{corpus}} \log p(y \mid x; \theta)$$

Luong et al. (2015)

Encoder-Decoder for Neural Machine Translation

Encoder: usually CNN, stacking RNN* with LSTM or GRU units...

* unidirectional (Luong et al. 2015) or bidirectional (Bahdanau et al. 2014).

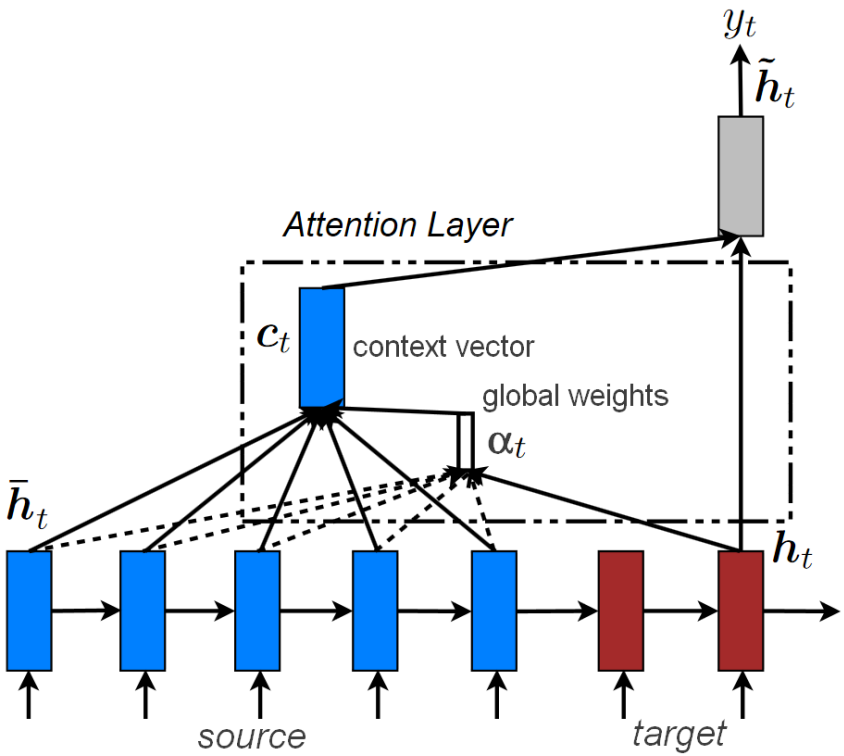
Decoder: unidirectional *RNN* (well suited to text generation), best if deep.

Generates the target sentence $(y_1, \ldots, y_{T_y})$ one word at a time:

$$P\left[y_t \mid \{y_1, \ldots, y_{t-1}\}, c_t\right] = \text{softmax}\left(W_s \tilde{h}_t\right)$$

Luong et al. (2015)

Global attention (all sentence)



$$P\Big[y_t \mid \{y_1, \ldots, y_{t-1}\}, c_t\Big] = \text{softmax}\Big(W_s \tilde{h}_t\Big)$$

attentional hidden state:

$$\tilde{h}_t = \tanh\Big(W_c[c_t; h_t]\Big)$$

(h_t: decoder hidden state) **context vector:**

$$c_t = \sum_{i=1}^{T_x} \alpha_{t,i}\, \bar{h}_i$$

($\bar{h}_i$: i^{th} encoder hidden state) **alignment vector:**

$$\alpha_{t,i} = \frac{\exp\Big(\text{score}(h_t, \bar{h}_i)\Big)}{\sum_{i'=1}^{T_x} \exp\Big(\text{score}(h_t, \bar{h}_{i'})\Big)}$$

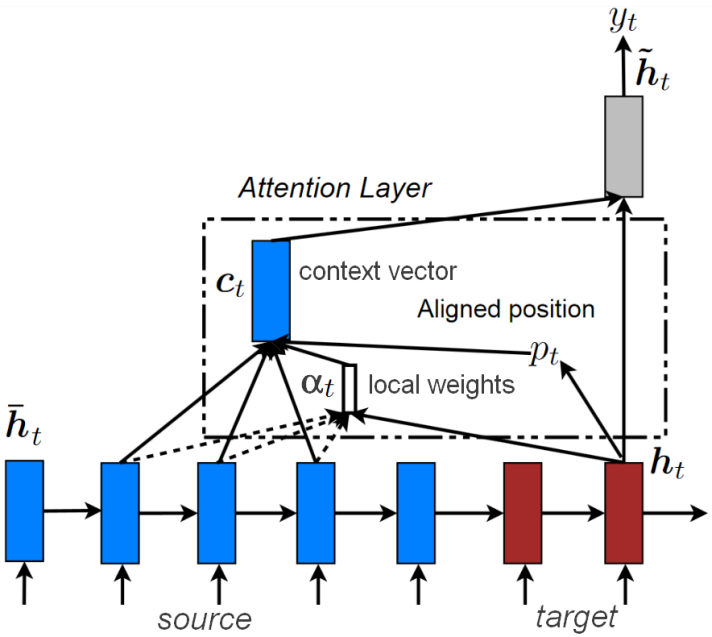
$$\text{score}(h_t, \bar{h}_i) = h_t^\top \bar{h}_i$$

Luong et al. (2015)

Local attention

(window length $2D$)

with predictive alignment



$$P\Big[y_t \mid y_{<t}, c_t\Big] = \text{softmax}\Big(W_s \tilde{h}_t\Big), \quad \tilde{h}_t = \tanh\Big(W_c[c_t; h_t]\Big)$$

aligned position (centre of the window):

$$p_t = T_x \cdot \sigma\Big(v_p^\top \tanh(W_p h_t)\Big)$$

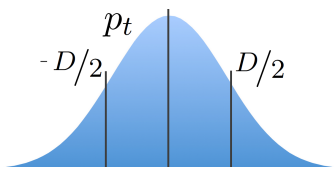
context vector:

$$c_t = \sum_{i=p_t-D}^{p_t+D} \alpha_{t,i}\, \bar{h}_i$$

alignment vector:

$$\alpha_{t,i} = \frac{\exp\Big(\text{score}(h_t, \bar{h}_i)\Big)}{\sum_{i'=p_t-D}^{p_t+D} \exp\Big(\text{score}(h_t, \bar{h}_{i'})\Big)} \exp\Big(-\frac{(i-p_t)^2}{2(D/2)^2}\Big)$$

$$\text{score}(h_t, \bar{h}_i) = h_t^\top W_\alpha \bar{h}_i$$



Luong et al. (2015)

Encoder-Decoder for Neural Machine Translation

Facts:

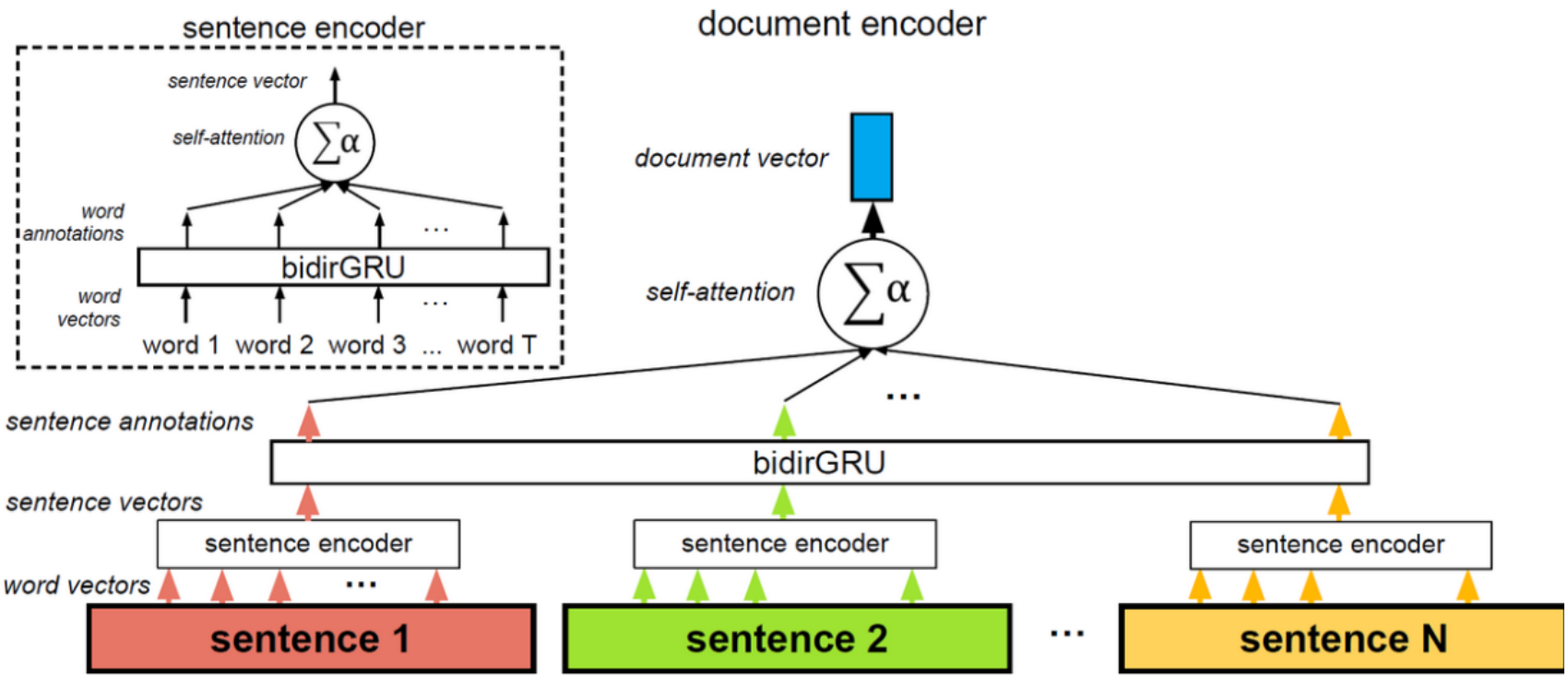
- Tested on the English $\leftrightarrow$ German task (WMT'14 dataset)
- 4.5M sentence pairs
- Encoder and decoder RNNs feature 4 layers of stacking and 1000-dimensional hidden states
- Window of size $D = 10$ for local attention
- Trained for 12 epochs (total of 7–10 days on a single GPU, at 1K target words/s)
- New state-of-the-art performance

Lessons learned:

- Local attention with predictive alignment gives better results than global attention
- Dot product ($\text{score}(h_t, \bar{h}_i) = h_t^\top \bar{h}_i$) works well for global attention
- The general formulation ($\text{score}(h_t, \bar{h}_i) = h_t^\top W_\alpha \bar{h}_i$) is better for local attention

Luong et al. (2015)

Self-attention for RNN encoders



Self-attention for RNN encoders

- Input is a sentence $(x_1, \ldots, x_T)$
- We're only interested in getting an embedding s of the sentence for some downstream task (e.g., classification)

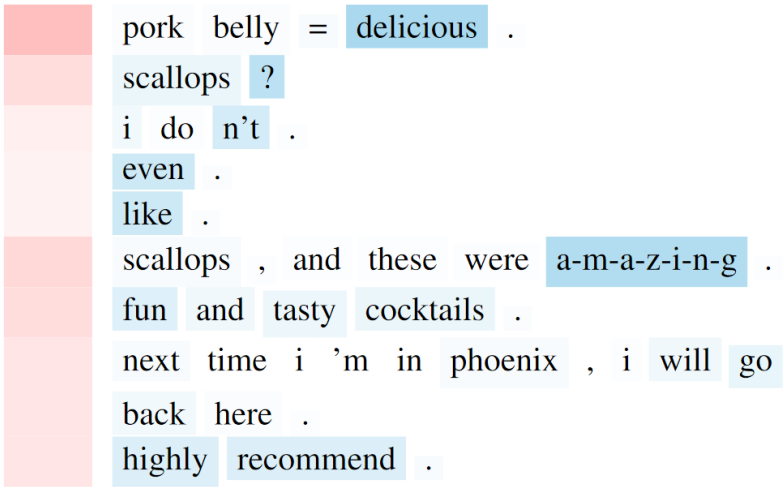
$$u_t = \tanh(Wh_t)$$

$$\alpha_t = \frac{\exp\left(\text{score}(u_t, u)\right)}{\sum_{t'=1}^{T} \exp\left(\text{score}(u_{t'}, u)\right)}$$

$$s = \sum_{t=1}^{T} \alpha_t h_t$$

where $\text{score}(u_t, u) = u_t^\top u$; h_t: encoder hidden state; u: context vector.

The same process can be repeated over the sentence vectors $s \rightarrow$ **hierarchical attention**



References I



I. D. Mienye, T. G. Swart, and G. Obaido,

"Recurrent neural networks: A comprehensive review of architectures, variants, and applications,"

Information, vol. 15, no. 9, p. 517, 2024.

https://doi.org/10.3390/info15090517



W. Yin, K. Kann, M. Yu, and H. Schütze,

"Comparative study of CNN and RNN for natural language processing,"

arXiv preprint arXiv:1702.01923, 2017.

https://arxiv.org/abs/1702.01923



J. Xiao and Z. Zhou,

"Research progress of RNN language model,"

in *Proc. IEEE Int. Conf. on Artificial Intelligence and Computer Applications (ICAICA)*, 2020, pp. 1285–1288.

https://doi.org/10.1109/ICAICA50127.2020.9182390